

Title Page

Lab 5 ARP Poisoning Forensics

Student Name: Athanasius Alekwe

Student ID: 2025/FWSD/11230

**Course Name: Basic Networking Skills for Digital
Forensics**

Instructor Name: Aminu Idris

Date of Submission: 17/09/2026

Executive Summary and Authorization

This practical focused on understanding how ARP works on a local network and how ARP traffic can be examined during a forensic investigation. The main aim was to observe normal ARP communication first and then analyze a supplied packet capture for signs that could indicate ARP poisoning.

The practical was carried out using Kali Linux inside an isolated VMware environment. I worked only with the systems and evidence provided for the lab and did not interact with any unauthorized or third-party network.

During the investigation, I captured normal ARP traffic, examined ARP request and reply packets, reviewed IP-to-MAC address relationships, and checked the supplied capture for suspicious or conflicting ARP information. I also verified the integrity of the supplied evidence using SHA-256 hashing.

The controlled live-poisoning section was not used as the main source of evidence because a final `controlled_arp_poison.pcapng` file was not available. Instead, the supplied `arp.pcap` file and my normal ARP capture were used for the analysis. The lab manual states that the live controlled section is optional and should only be performed inside an approved host-only or internal network.

Laboratory Environment

The practical was carried out on a Kali Linux virtual machine running in VMware Workstation.

During the final stage of the lab, the Kali system was connected to an isolated network using the following address:

192.168.40.128/24

The active network interface was:

eth0

The final routing information showed that the machine was connected to the local 192.168.40.0/24 network.

There was no normal Internet default route shown in the final routing table. This was suitable for the practical because the lab instructions require the exercise to remain isolated from production, office, school, public, or other unauthorized networks.

Evidence Acquisition and Integrity

The main evidence files used during the practical were:

evidence/arp.pcap

evidence/normal_arp.pcapng

working/arp_working.pcap

The original supplied ARP capture was kept inside the evidence folder, while a separate copy was placed in the working folder for analysis.

This was important because the original evidence should not be changed during an investigation.

SHA-256 hashes were calculated for the evidence files and saved in the reports folder.

The hash results confirmed that the original ARP capture and the working copy were identical. This gave confidence that the evidence had not been modified during the analysis.

The lab manual specifically requires the original capture to be preserved, analysis to be carried out on a working copy, and SHA-256 hashes to be recorded.

Normal ARP Behaviour

Before examining the supplied evidence, I first observed how normal ARP communication works.

A normal ARP capture was created and saved as:

evidence/normal_arp.pcapng

The ARP packet fields were also exported to:

reports/normal_arp_fields.tsv

In normal ARP communication, a host sends an ARP request when it needs to find the MAC address associated with a particular IPv4 address.

A normal ARP request uses:

ARP opcode 1

and is usually sent to the broadcast MAC address:

ff:ff:ff:ff:ff:ff

The device that owns the requested IP address then responds with an ARP reply.

A normal ARP reply uses:

ARP opcode 2

The reply tells the requesting device which MAC address belongs to the requested IP address.

This part of the practical helped me understand what normal ARP traffic looks like before checking for suspicious behaviour. The lab manual also identifies opcode 1 as an ARP request and opcode 2 as an ARP reply.

Analysis of the Supplied ARP Capture

The supplied ARP evidence was analyzed using:

working/arp_working.pcap

The capture contained eight ARP frames.

The most important request and reply sequence was:

Frame	Opcode	Sender IP	Sender MAC	Target IP	Interpretation
3	1	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	ARP Request
4	2	136.160.215.194	00:50:56:86:02:65	136.160.215.15	ARP Reply
5	1	136.160.215.194	00:50:56:86:02:65	136.160.215.15	ARP Request
6	2	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	ARP Reply

IP-to-MAC Address Analysis

The ARP IP-to-MAC summary produced the following results:

4 136.160.215.1 00:1b:17:00:0a:30

2 136.160.215.194 00:50:56:86:02:65

2 136.160.215.15 00:50:56:86:cb:fc

The results showed that each IP address remained associated with the same MAC address throughout the ARP frames that were examined.

For example:

136.160.215.1

was consistently associated with:

00:1b:17:00:0a:30

Likewise:

136.160.215.194

was consistently associated with:

00:50:56:86:02:65

and:

136.160.215.15

was consistently associated with:

00:50:56:86:cb:fc

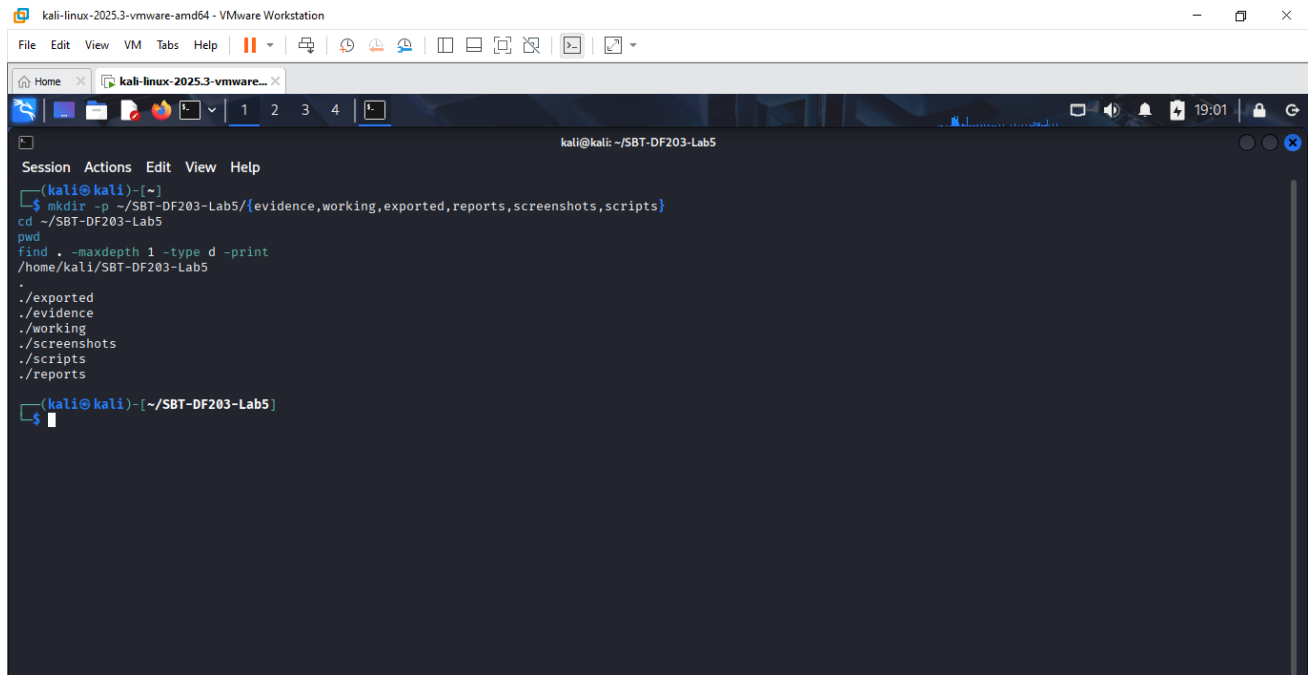
I did not observe one IP address changing between different MAC addresses in the analyzed ARP frames.

I also did not observe one MAC address claiming both 136.160.215.15 and 136.160.215.194.

Because of this, the ARP frames examined did not provide clear evidence of a conflicting IP-to-MAC mapping.

Practical Evidence

Create the forensic lab folders



The screenshot shows a Kali Linux terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal displays the following commands and output:

```
(kali@kali)-[~]
$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab5
pwd
/home/kali/SBT-DF203-Lab5
find . -maxdepth 1 -type d -print
./exported
./evidence
./working
./screenshots
./scripts
./reports
(kali@kali)-[~/SBT-DF203-Lab5]
```

Fig 1.0 Lab folder structure and start time.

Evidence field	Entry
Case/Lab Identifier	SBT-DF203-Lab5
Trainee Name	Athanasius Alekwe
Registration Number	2025/FWSD/11230
Evidence file	arp.pcap
Working copy	working/arp_working.pcap

Explanation

This screenshot shows the creation of the Lab 5 folder structure used to organise the practical evidence, working files, reports, screenshots, and scripts. Setting up the folders at the beginning helped me keep the original evidence separate from the files used during analysis.

The screenshot shows a terminal window in a Kali Linux virtual machine. The user has created a directory structure under `~/SBT-DF203-Lab5` and then used `dpkg -l` to check the status of installed packages. The output shows that `net-tools`, `python3-scapy`, `tshark`, and `wireshark` are all installed.

```
(kali@kali)~[~/SBT-DF203-Lab5]
$ mkdir -p ~/SBT-DF203-Lab5/{evidence,working,exported,reports,screenshots,scripts}
cd ~/SBT-DF203-Lab5
pwd
/home/kali/SBT-DF203-Lab5
find . -maxdepth 1 -type d -print
./exported
./evidence
./working
./screenshots
./scripts
./reports

(kali@kali)~[~/SBT-DF203-Lab5]
$ dpkg -l wireshark tshark python3-scapy net-tools
Desired=Unknown/Install/Remove/Purge/Hold
Status=Not/Inst/Conf-files/Unpacked/half-Inst/Half-conf/Half-inst/trig-aWait/Trig-pend
Err?=(none)/Reinst-required (Status,Err: uppercase=bad)
Name Version Architecture Description
ii net-tools 2.10-1.3 amd64 NET-3 networking toolkit
ii python3-scapy 2.6.1+dfsg-1 all Packet generator/sniffer and network scanner/discovery (Python 3)
ii tshark 4.4.7-1+b1 amd64 network traffic analyzer - console version
ii wireshark 4.4.7-1+b1 amd64 network traffic analyzer - graphical interface

(kali@kali)~[~/SBT-DF203-Lab5]
```

Fig 1.1 Confirming all package are installed.

Explanation

This screenshot confirms that the main tools needed for the practical were already available on the Kali Linux system. Checking the packages first helped me avoid reinstalling tools unnecessarily and confirmed that the environment was ready for the ARP investigation.

The screenshot shows the same terminal window as Figure 1.1, but now the user has run `sudo apt update`. The output shows the progress of updating the package lists from various sources, including Google Linux, Kali Linux, and Kali Rolling. The progress bar at the bottom indicates that 27% of the packages have been updated.

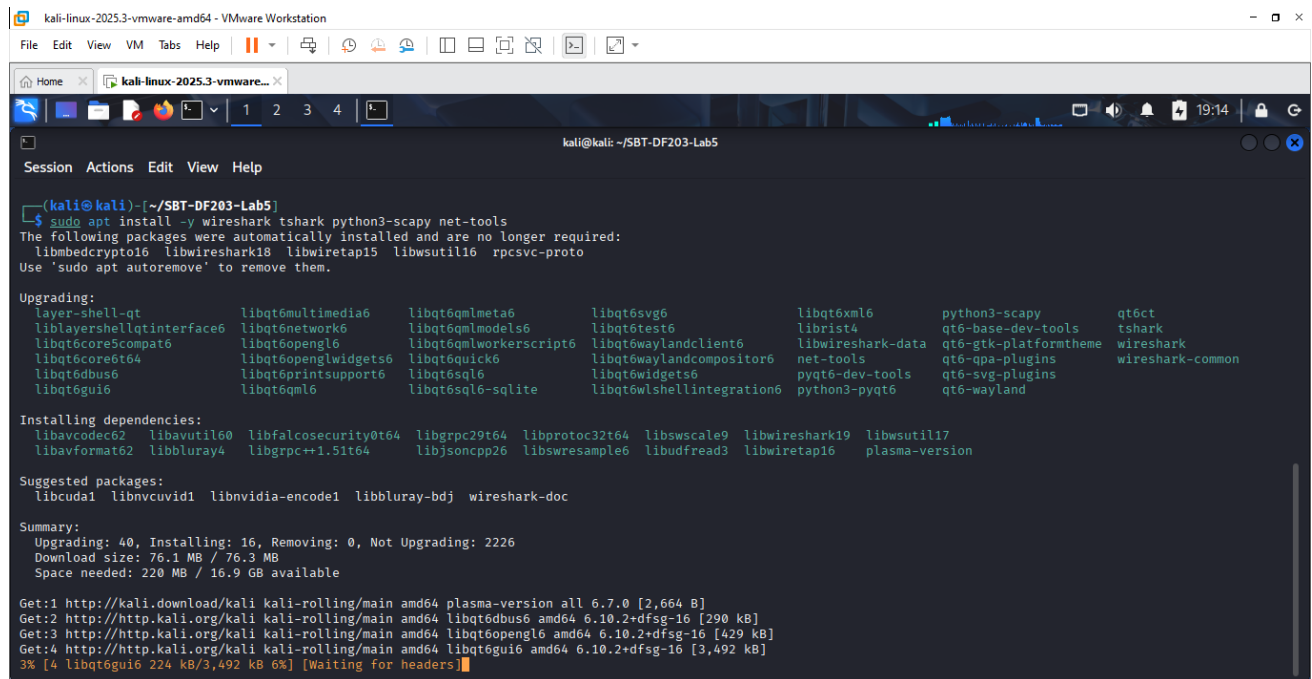
```
(kali@kali)~[~/SBT-DF203-Lab5]
$ sudo apt update
[sudo] password for kali:
Get:2 https://dl.google.com/linux/chrome/deb stable InRelease [2,548 B]
Get:3 https://dl.google.com/linux/chrome/deb stable/main amd64 Packages [1,409 B]
Get:1 http://kali.download/kali kali-rolling InRelease [34.0 kB]
Get:4 http://kali.download/kali kali-rolling/main amd64 Packages [21.6 MB]
27% [4 Packages 10.9 MB/21.6 MB 50%]
680 kB/s 1min 37s
```

Fig 1.2 Updating the applications.

Explanation

This screenshot shows the package update process being started on Kali Linux. I performed this step to make

sure the system had access to the current package information before continuing with the practical.



```
kali@kali: ~/SBT-DF203-Lab5
(kali@kali)~$ sudo apt install -y wireshark tshark python3-scapy net-tools
The following packages were automatically installed and are no longer required:
  libmbcrypted016 libwireshark18 libwireshark15 libwsutil16 rpcsvc-proto
Use 'sudo apt autoremove' to remove them.

Upgrading:
  layer-shell-qt libqt6multimedia6 libqt6qmlmeta6 libqt6svg6 libqt6xml6 python3-scapy qt6ct
  liblayershellqtinterface6 libqt6network6 libqt6qmlmodels6 libqt6test6 librist4 qt6-base-dev-tools tshark
  libqt6core5compat6 libqt6opengl6 libqt6qmlworkerscript6 libqt6waylandclient6 libwireshark-data qt6-gtk-platformtheme wireshark
  libqt6core6t64 libqt6openglwidgets6 libqt6quick6 libqt6waylandcompositor6 net-tools qt6-qpas-plugins wireshark-common
  libqt6dbus6 libqt6printsupport6 libqt6sql6 libqt6widgets6 pyqt6-dev-tools qt6-svg-plugins qt6-wayland
  libqt6gui6 libqt6qml6 libqt6sql6-sqlite libqt6wslshellintegration6 python3-pyqt6 qt6-wayland

Installing dependencies:
  libavcodec62 libavutil60 libfalcosecurity0t64 libgrpc29t64 libprotoc32t64 libswscale0 libwireshark19 libwsutil17
  libavformat62 libbluray4 libgrpc++1.51t64 libjsoncpp26 libswresample6 libudfread3 libwireshark16 plasma-version

Suggested packages:
  libcuda1 libnvcuvid1 libnvidia-encode1 libbluray-bdj wireshark-doc

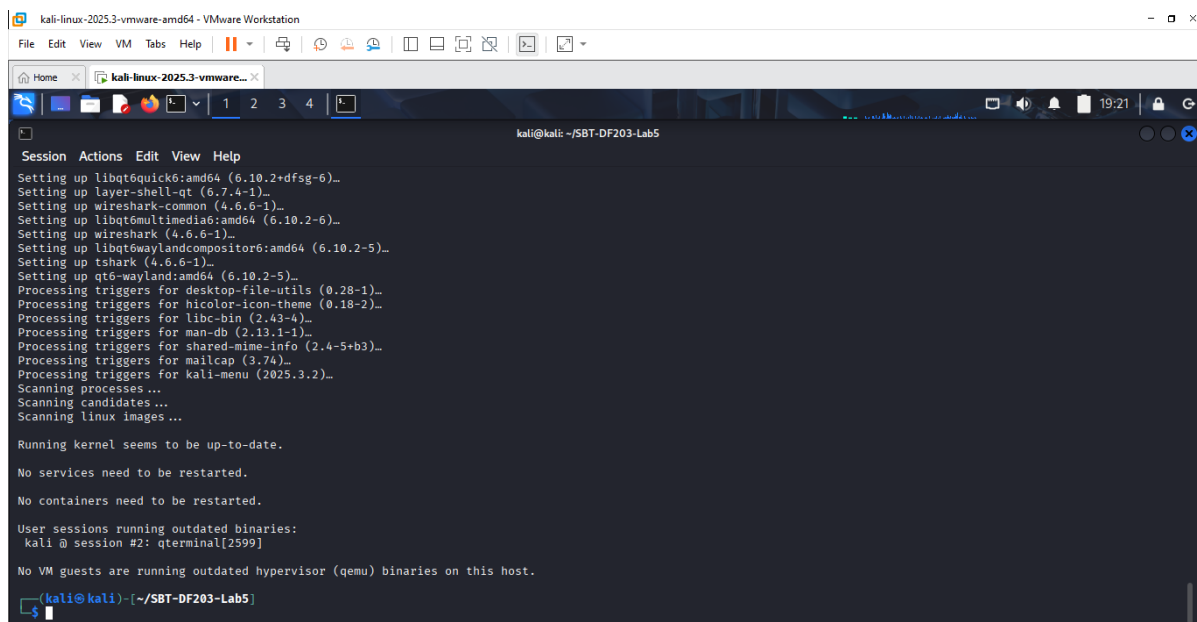
Summary:
  Upgrading: 40, Installing: 16, Removing: 0, Not Upgrading: 2226
  Download size: 76.1 MB / 76.3 MB
  Space needed: 220 MB / 16.9 GB available

Get:1 http://kali.download/kali kali-rolling/main amd64 plasma-version all 6.7.0 [2,664 B]
Get:2 http://http.kali.org/kali kali-rolling/main amd64 libqt6dbus6 amd64 6.10.2+dfsg-16 [290 kB]
Get:3 http://http.kali.org/kali kali-rolling/main amd64 libqt6opengl6 amd64 6.10.2+dfsg-16 [429 kB]
Get:4 http://http.kali.org/kali kali-rolling/main amd64 libqt6gui6 amd64 6.10.2+dfsg-16 [3,492 kB]
3% [4 libqt6gui6 224 kB/3,492 kB 6%] [Waiting for headers]
```

Fig 1.3 Updating the applications page 1.

Explanation

This screenshot continues the system update process and shows Kali retrieving and processing package information. It confirms that the update command was running normally without interrupting the lab setup.



```
kali@kali: ~/SBT-DF203-Lab5
(kali@kali)~$ sudo apt install -y wireshark tshark python3-scapy net-tools
Setting up libqt6quick6:amd64 (6.10.2+dfsg-6)...
Setting up layer-shell-qt (6.7.4-1)...
Setting up wireshark-common (4.6.6-1)...
Setting up libqt6multimedia6:amd64 (6.10.2-6)...
Setting up wireshark (4.6.6-1)...
Setting up libqt6waylandcompositor6:amd64 (6.10.2-5)...
Setting up tshark (4.6.6-1)...
Setting up qt6-wayland:amd64 (6.10.2-5)...
Processing triggers for desktop-file-utils (0.28-1)...
Processing triggers for hicolor-icon-theme (0.18-2)...
Processing triggers for libc-bin (2.43-4)...
Processing triggers for man-db (2.13.1-1)...
Processing triggers for shared-mime-info (2.4-5+b3)...
Processing triggers for mailcap (3.74)...
Processing triggers for kali-menu (2025.3.2)...
Scanning processes...
Scanning candidates...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

User sessions running outdated binaries:
kali @ session #2: qterminal[2599]

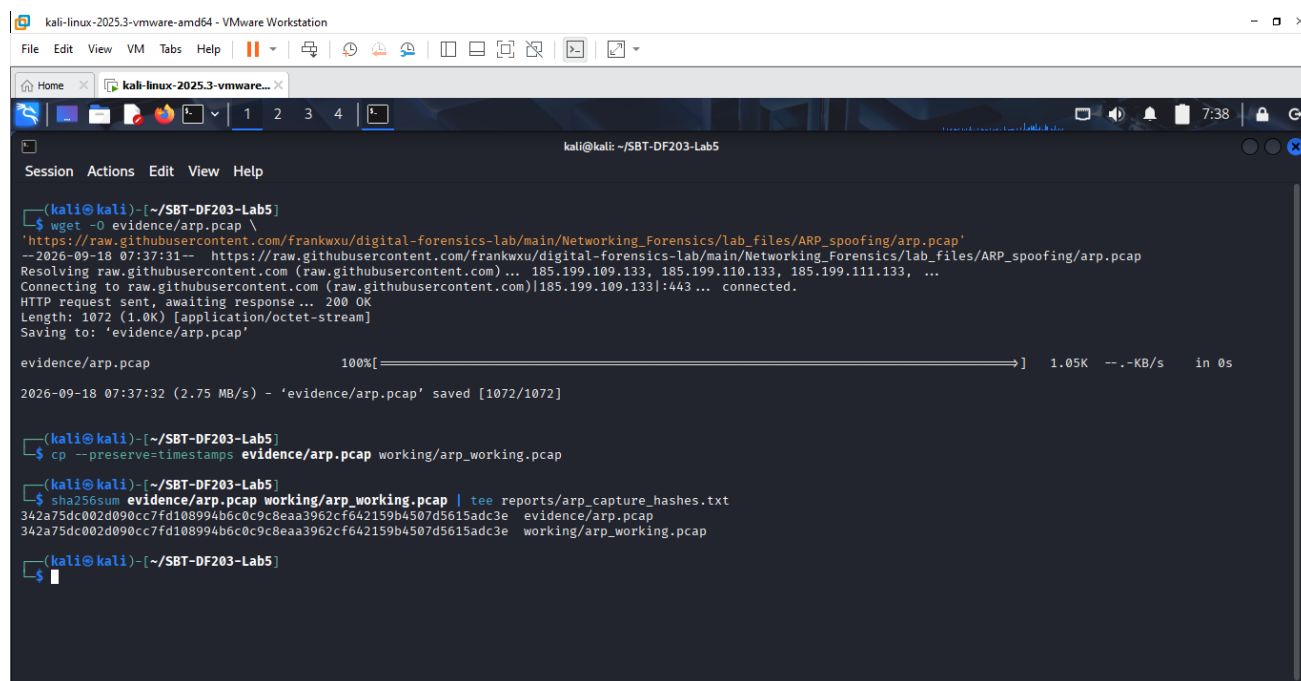
No VM guests are running outdated hypervisor (qemu) binaries on this host.

(kali@kali)~$
```

Fig 1.4 Updating the applications page 2.

Explanation

This screenshot shows the final part of the update process before moving on to the evidence work. Completing this step helped ensure that the analysis environment was prepared before I started handling the ARP capture.



```
(kali@kali)~[/SBT-DF203-Lab5]
$ wget -O evidence/arp.pcap \
'https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap'
--2026-09-18 07:37:31-- https://raw.githubusercontent.com/frankwxu/digital-forensics-lab/main/Networking_Forensics/lab_files/ARP_spoofing/arp.pcap
Resolving raw.githubusercontent.com (raw.githubusercontent.com)... 185.199.109.133, 185.199.110.133, 185.199.111.133, ...
Connecting to raw.githubusercontent.com (raw.githubusercontent.com)|185.199.109.133|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 1072 (1.0K) [application/octet-stream]
Saving to: 'evidence/arp.pcap'

evidence/arp.pcap      100%[=====] 1.05K --.-KB/s in 0s

2026-09-18 07:37:32 (2.75 MB/s) - 'evidence/arp.pcap' saved [1072/1072]

(kali@kali)~[/SBT-DF203-Lab5]
$ cp --preserve=timestamps evidence/arp.pcap working/arp_working.pcap

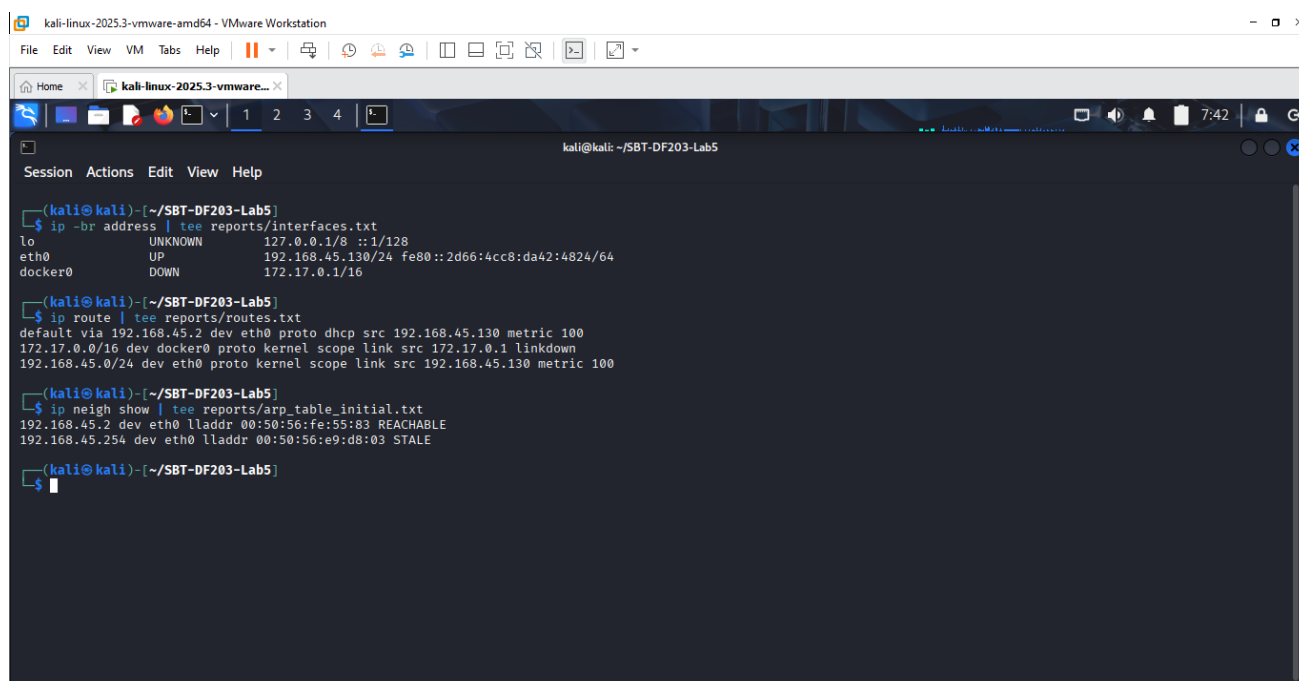
(kali@kali)~[/SBT-DF203-Lab5]
$ sha256sum evidence/arp.pcap working/arp_working.pcap | tee reports/arp_capture_hashes.txt
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e evidence/arp.pcap
342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e working/arp_working.pcap

(kali@kali)~[/SBT-DF203-Lab5]
```

Fig 1.5 Acquiring and preserve and hashing the supplied ARP evidence.

Explanation

This screenshot shows the supplied arp.pcap being preserved as the original evidence and copied to the working directory for analysis. SHA-256 hashes were also calculated so I could confirm that the working copy was identical to the original evidence.



```
(kali@kali)~[/SBT-DF203-Lab5]
$ ip -br address | tee reports/interfaces.txt
lo UNKNOWN 127.0.0.1/8 ::1/128
eth0 UP 192.168.45.130/24 fe80::2d66:4cc8:da42:4824/64
docker0 DOWN 172.17.0.1/16

(kali@kali)~[/SBT-DF203-Lab5]
$ ip route | tee reports/routes.txt
default via 192.168.45.2 dev eth0 proto dhcp src 192.168.45.130 metric 100
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.45.0/24 dev eth0 proto kernel scope link src 192.168.45.130 metric 100

(kali@kali)~[/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_initial.txt
192.168.45.2 dev eth0 lladdr 00:50:56:fe:55:83 REACHABLE
192.168.45.254 dev eth0 lladdr 00:50:56:e9:d8:03 STALE

(kali@kali)~[/SBT-DF203-Lab5]
```

Fig 1.6 Recording the initial network state.

Explanation

This screenshot records the network interface, routing information, and ARP or neighbour state before the

main analysis began. Capturing this information provided a baseline that could later be compared with the network state after the practical.

```

kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
(kali@kali)-[~/SBT-DF203-Lab5]
$ date
Thu Sep 17 09:13:07 AM WAT 2026
(kali@kali)-[~/SBT-DF203-Lab5]
$ hostname
kali
(kali@kali)-[~/SBT-DF203-Lab5]
$ cat /etc/os-release | head -n 3
PRETTY_NAME="Kali GNU/Linux Rolling"
NAME="Kali GNU/Linux"
VERSION_ID="2025.3"
(kali@kali)-[~/SBT-DF203-Lab5]
$

```

Fig 1.7 Recording the Mini Evidence / Chain-of-Custody information.

Field	Student Entry
Case/Lab Identifier	SBT-DF203-Lab5-Alekwe_Athanasius
Trainee Name	Alekwe Athanasius
Date and Time Started	Thu Sep 17 09:13:07 AM WAT 2026
Evidence File Name(s)	arp.pcap, arp_working.pcap
Source / Generation Method	Supplied authorised ARP training PCAP downloaded from the course-provided GitHub source. A forensic working copy was created using <code>cp --preserve=timestamps</code> .
Original SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
Working-Copy SHA-256	342a75dc002d090cc7fd108994b6c0c9c8eaa3962cf642159b4507d5615adc3e
Analysis Workstation / VM	VMware virtual machine; hostname kali; Kali GNU/Linux Rolling 2025.3
Notes on Changes	Original evidence preserved in evidence/. Identical working copy stored in working/. SHA-256 hashes match. No modification to the original PCAP was made.

Explanation

This screenshot documents the case identifier, trainee details, evidence files, SHA-256 hashes, workstation

information, and the method used to obtain the evidence. The matching original and working-copy hashes show that the supplied PCAP remained unchanged during preparation.

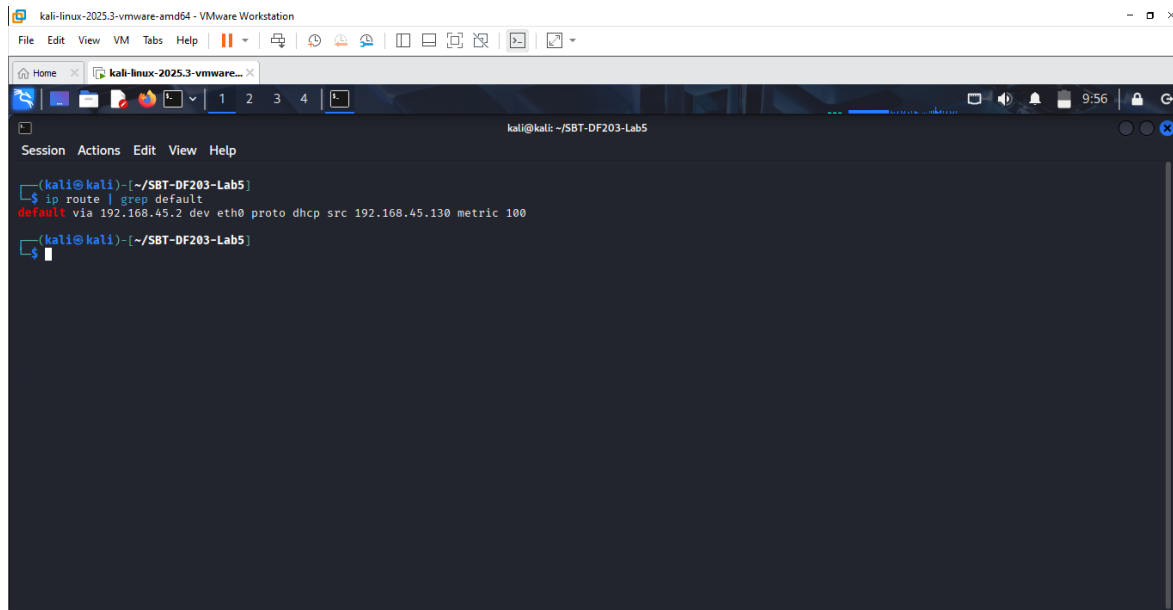


Fig 1.8 Normal ARP Resolution.

Explanation

This screenshot shows the preparation for observing normal ARP communication on the network. The gateway and interface information were identified so that a normal ARP request-and-reply exchange could be captured and examined.

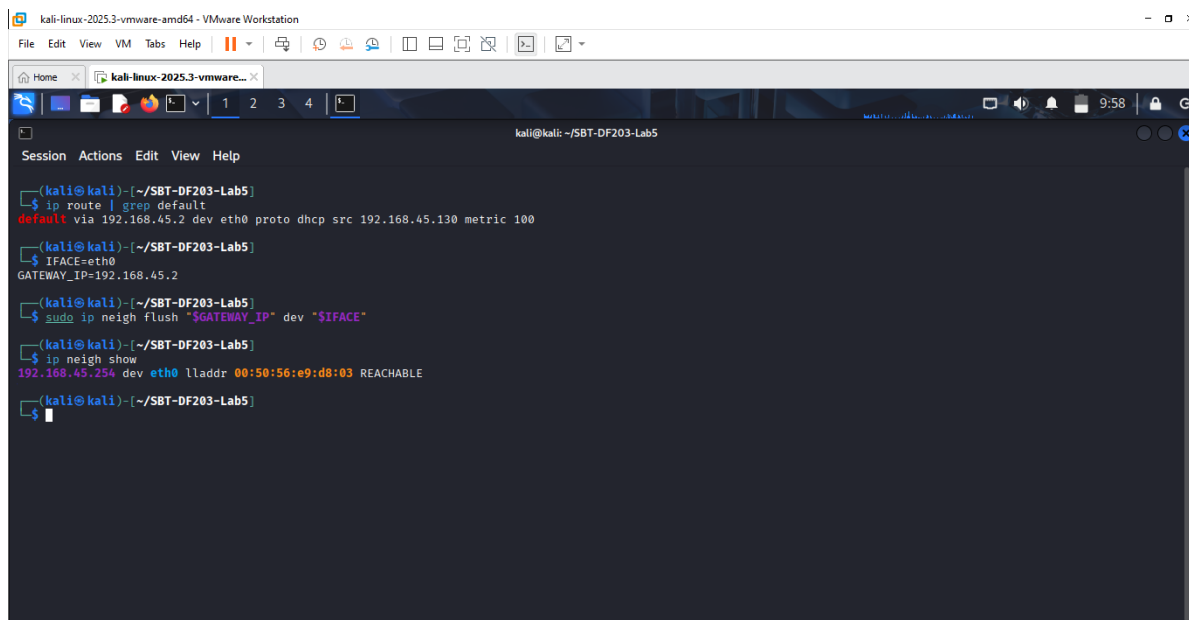
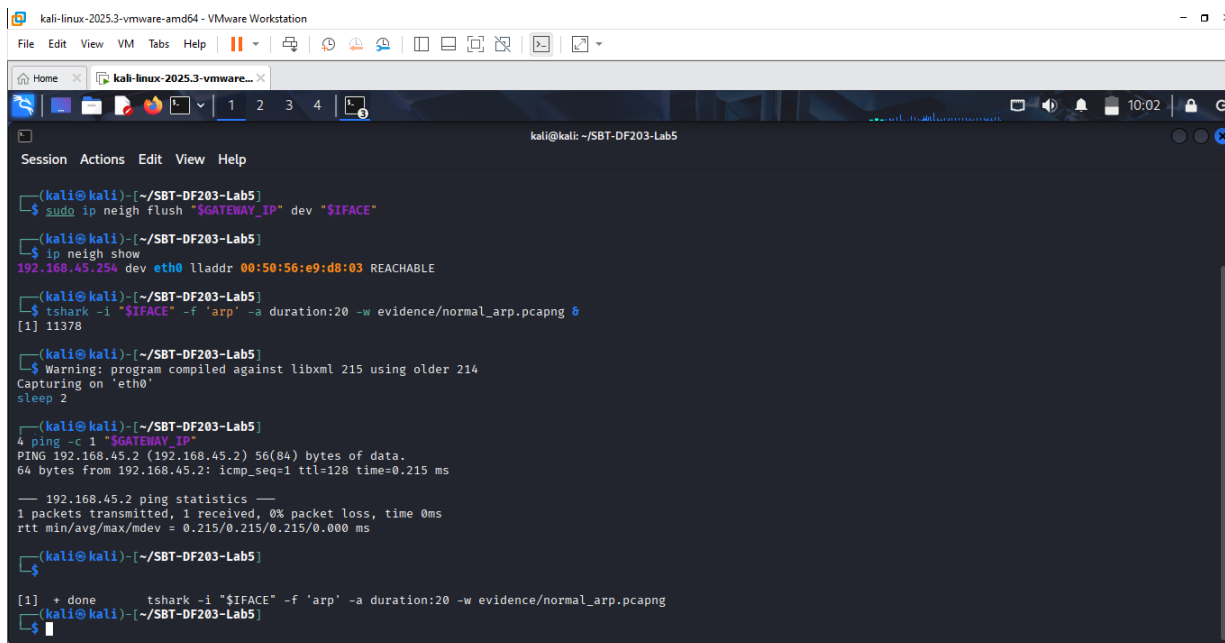


Fig 1.9 Capturing normal ARP resolution and clearing only that gateway entry.

Explanation

This screenshot shows the specific neighbour entry being cleared before the ARP capture was started. Doing

this forced the system to resolve the address again, allowing the normal ARP request and reply to appear in the capture.



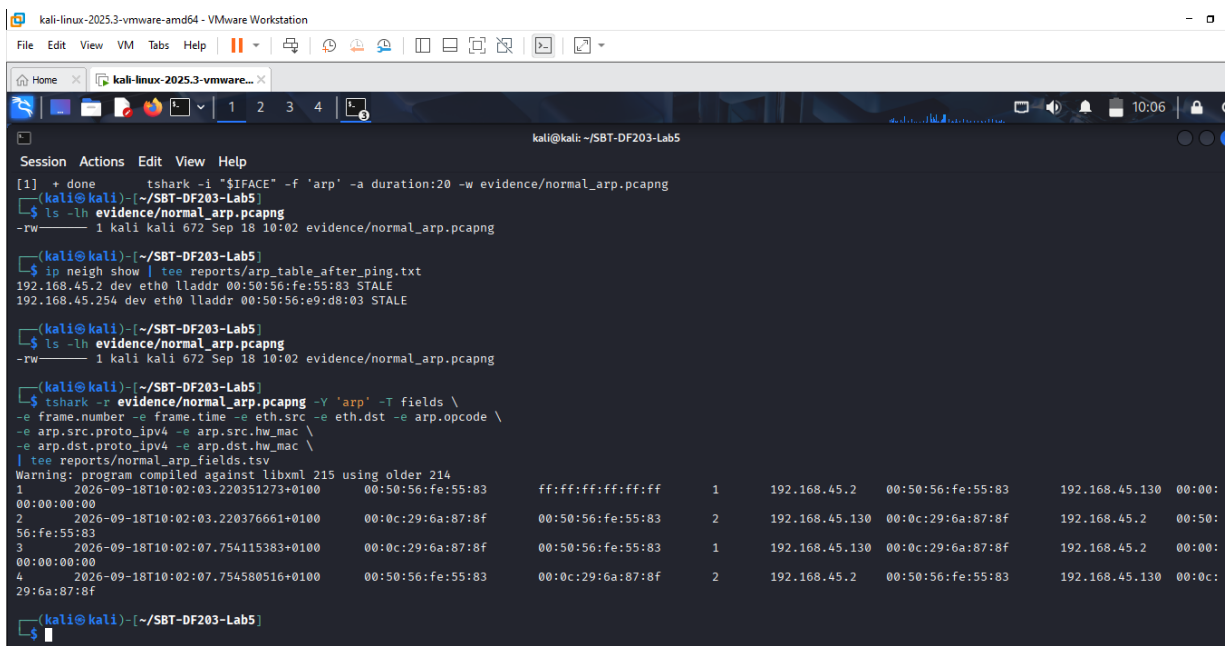
```
kali@kali: ~/SBT-DF203-Lab5
$ sudo ip neigh flush "$GATEWAY_IP" dev "$IFACE"
$ ip neigh show
192.168.45.254 dev eth0 lladdr 00:50:56:e9:d8:03 REACHABLE
$ tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng &
[1] 11378
$ Warning: program compiled against libxml 215 using older 214
Capturing on 'eth0'
sleep 2
$ ping -c 1 "$GATEWAY_IP"
PING 192.168.45.2 (192.168.45.2) 56(84) bytes of data:
64 bytes from 192.168.45.2: icmp_seq=1 ttl=128 time=0.215 ms

--- 192.168.45.2 ping statistics ---
1 packets transmitted, 1 received, 0% packet loss, time 0ms
rtt min/avg/max/mdev = 0.215/0.215/0.215/0.000 ms
$
[1] + done tshark -i "$IFACE" -f 'arp' -a duration:20 -w evidence/normal_arp.pcapng
$
```

Fig 2.0 Capture succeeded.

Explanation

This screenshot confirms that the normal ARP capture completed successfully and produced the expected ARP traffic. The capture contained four frames showing communication between Kali and the gateway, which could then be examined in detail.



```
kali@kali: ~/SBT-DF203-Lab5
$ ls -lh evidence/normal_arp.pcapng
-rw-r--r-- 1 kali kali 672 Sep 18 10:02 evidence/normal_arp.pcapng
$ ip neigh show | tee reports/arp_table_after_ping.txt
192.168.45.2 dev eth0 lladdr 00:50:56:fe:55:83 STALE
192.168.45.254 dev eth0 lladdr 00:50:56:e9:d8:03 STALE
$ ls -lh evidence/normal_arp.pcapng
-rw-r--r-- 1 kali kali 672 Sep 18 10:02 evidence/normal_arp.pcapng
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/normal_arp_fields.tsv
Warning: program compiled against libxml 215 using older 214
1 2026-09-18T10:02:03.220351273+0100 00:50:56:fe:55:83 ff:ff:ff:ff:ff:ff 1 192.168.45.2 00:50:56:fe:55:83 192.168.45.130 00:00:00:00:00:00
2 2026-09-18T10:02:03.220376661+0100 00:0c:29:6a:87:8f 00:50:56:fe:55:83 2 192.168.45.130 00:0c:29:6a:87:8f 192.168.45.2 00:50:56:fe:55:83
3 2026-09-18T10:02:07.754115383+0100 00:0c:29:6a:87:8f 00:50:56:fe:55:83 1 192.168.45.130 00:0c:29:6a:87:8f 192.168.45.2 00:00:00:00:00:00
4 2026-09-18T10:02:07.754580516+0100 00:50:56:fe:55:83 00:0c:29:6a:87:8f 2 192.168.45.2 00:50:56:fe:55:83 192.168.45.130 00:0c:29:6a:87:8f
$
```

Fig 2.0.1 Capture succeeded page 1.

What the four captured frames mean

Frame	Opcode	Source	→ Destination	Interpretation
1	1		Gateway → Broadcast	Gateway asks: Who has 192.168.45.130?
2	2		Kali → Gateway	Kali replies: 192.168.45.130 is at 00:0c:29:6a:87:8f
3	1		Kali → Gateway	Kali asks/validates: Who has 192.168.45.2?
4	2		Gateway → Kali	Gateway replies: 192.168.45.2 is at 00:50:56:fe:55:83

Frame 1 — Normal broadcast ARP request

Ethernet source: 00:50:56:fe:55:83
Ethernet destination: ff:ff:ff:ff:ff:ff
ARP opcode: 1
Sender IP: 192.168.45.2
Sender MAC: 00:50:56:fe:55:83
Target IP: 192.168.45.130

Frame 2 — Normal ARP reply

Ethernet source: 00:0c:29:6a:87:8f
Ethernet destination: 00:50:56:fe:55:83
ARP opcode: 2
Sender IP: 192.168.45.130
Sender MAC: 00:0c:29:6a:87:8f
Target IP: 192.168.45.2
Target MAC: 00:50:56:fe:55:83

Explanation

This screenshot confirms that the normal ARP capture completed successfully and produced the expected ARP traffic. The capture contained four frames showing communication between Kali and the gateway, which could then be examined in detail.

```
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali@kali: ~/SBT-DF203-Lab5
$ tshark -r evidence/normal_arp.pcapng -Y 'arp' -T fields \
-e frame.number -e frame.time -e eth.src -e eth.dst -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/normal_arp_fields.tsv
Warning: program compiled against libxml 215 using older 214
1 2026-09-18T10:02:03.220351273+0100 00:50:56:fe:55:83 ff:ff:ff:ff:ff:ff 1 192.168.45.2 00:50:56:fe:55:83 192.168.45.130 00:00:
00:00:00:00
2 2026-09-18T10:02:03.220376661+0100 00:0c:29:6a:87:8f 00:50:56:fe:55:83 2 192.168.45.130 00:0c:29:6a:87:8f 192.168.45.2 00:50:
56:fe:55:83
3 2026-09-18T10:02:07.754115383+0100 00:0c:29:6a:87:8f 00:50:56:fe:55:83 1 192.168.45.130 00:0c:29:6a:87:8f 192.168.45.2 00:00:
00:00:00:00
4 2026-09-18T10:02:07.754580516+0100 00:50:56:fe:55:83 00:0c:29:6a:87:8f 2 192.168.45.2 00:50:56:fe:55:83 192.168.45.130 00:0c:
29:6a:87:8f
(kali@kali)~/SBT-DF203-Lab5
$ sha256sum evidence/normal_arp.pcapng | tee reports/normal_arp_hash.txt
0e614d341351724285cbf87ff3b267be5bf1344fda90da21af57fc896260f4d5 evidence/normal_arp.pcapng
(kali@kali)~/SBT-DF203-Lab5
$ ls -lh evidence/normal_arp.pcapng \
reports/normal_arp_fields.tsv \
reports/arp_table_after_ping.txt \
reports/normal_arp_hash.txt
-rw-r--r-- 1 kali kali 672 Sep 18 10:02 evidence/normal_arp.pcapng
-rw-rw-r-- 1 kali kali 110 Sep 18 10:05 reports/arp_table_after_ping.txt
-rw-rw-r-- 1 kali kali 556 Sep 18 10:06 reports/normal_arp_fields.tsv
-rw-rw-r-- 1 kali kali 93 Sep 18 10:11 reports/normal_arp_hash.txt
(kali@kali)~/SBT-DF203-Lab5
$
```

Fig 2.1 Calculating the hash and confirming the evidence outputs.

Explanation

This screenshot shows the normal ARP capture being hashed and the generated evidence files being checked. The hash provides an integrity record for the capture, while the file check confirms that the required output was successfully created.

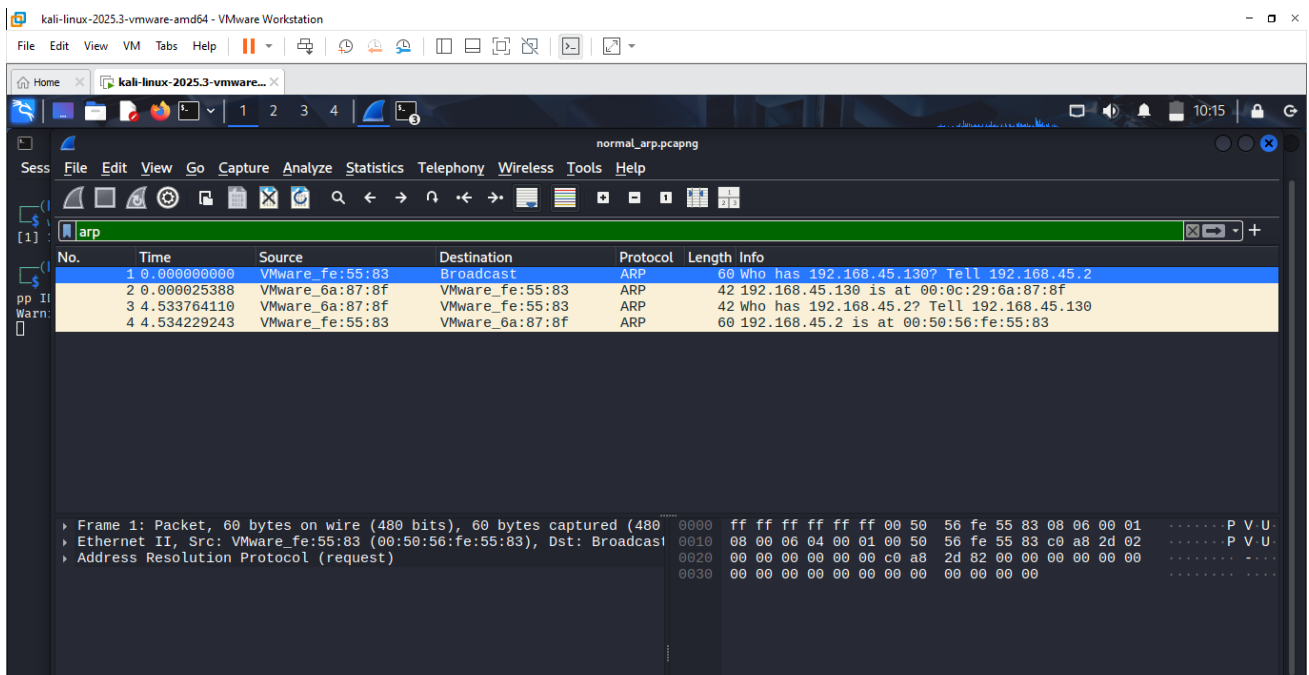


Fig 2.2 Opening the capture with wireshark.

Explanation

This screenshot shows the normal ARP capture opened in Wireshark for packet-level examination. Wireshark

made it possible to inspect the Ethernet and ARP fields of the captured request and reply frames.

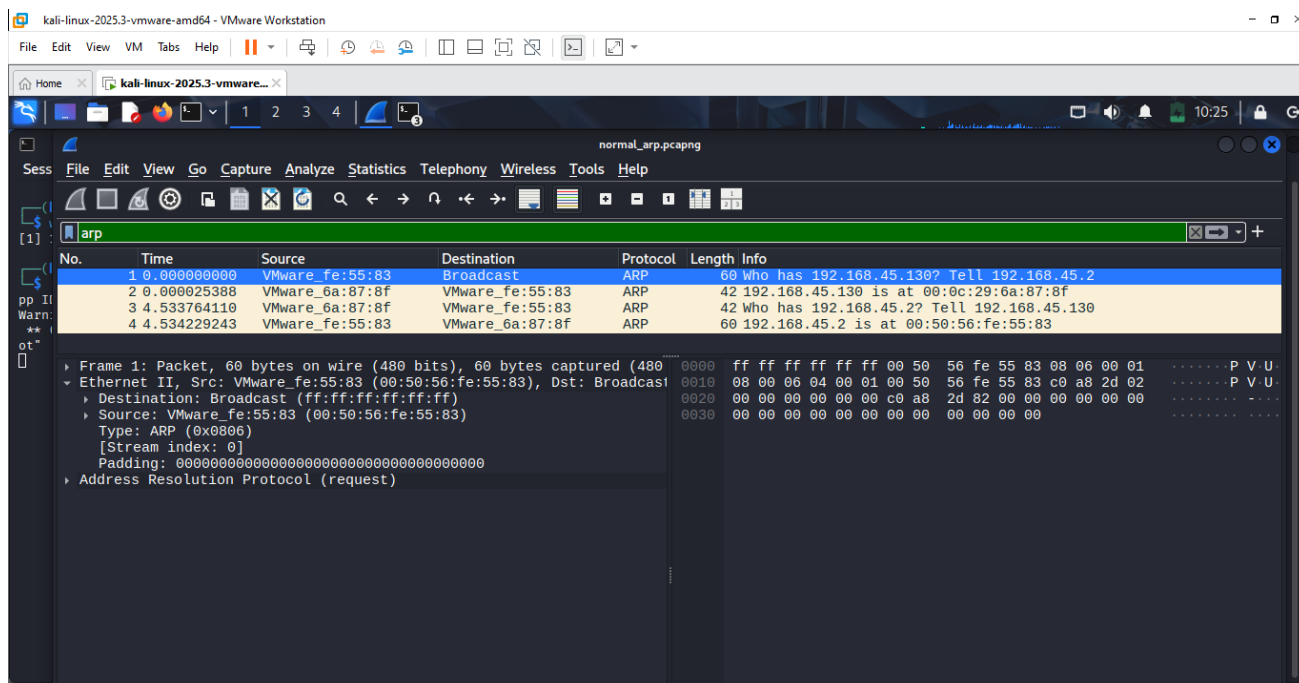


Fig 2.3 Taking record of Frame 1 Ethernet.

Explanation

This screenshot focuses on the Ethernet information for Frame 1. The frame was sent from the gateway MAC address to the broadcast address ff:ff:ff:ff:ff:ff, which is expected for a normal ARP request.

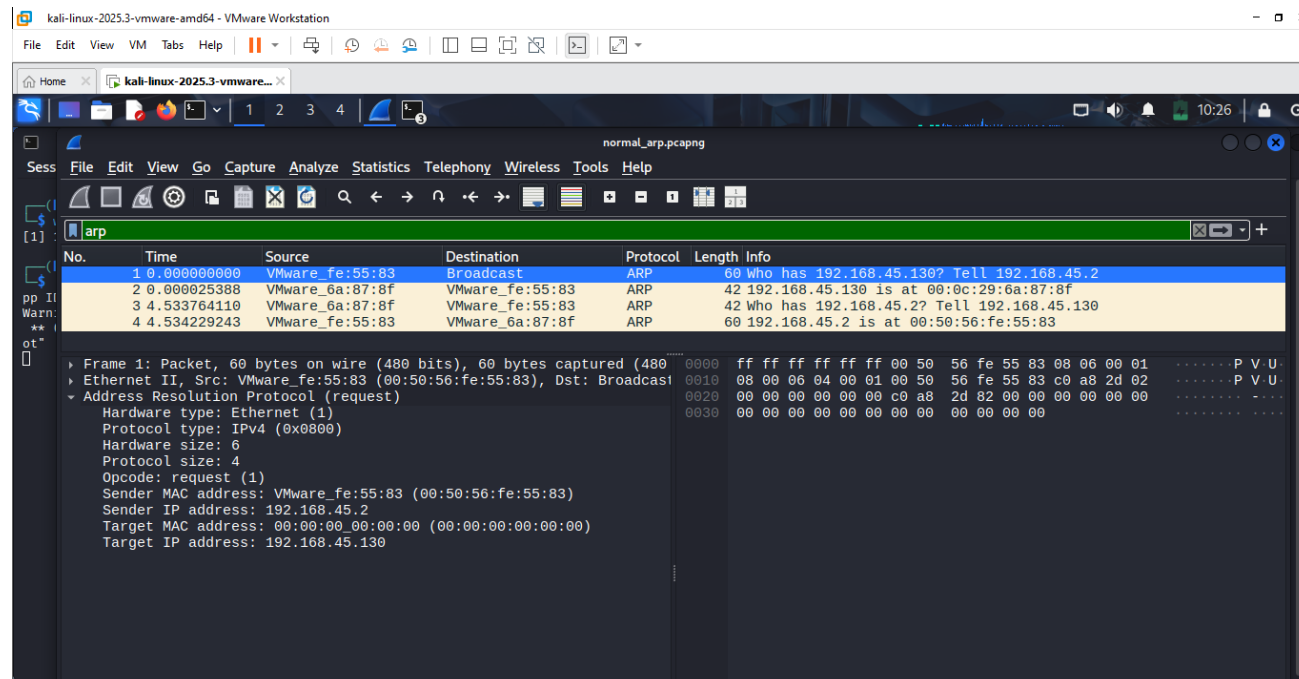


Fig 2.4 Taking record of Frame 1 Address Resolution Protocol.

Explanation

This screenshot shows the ARP fields contained in Frame 1. The packet used opcode 1, with the gateway asking which device owned 192.168.45.130, confirming that it was an ARP request.

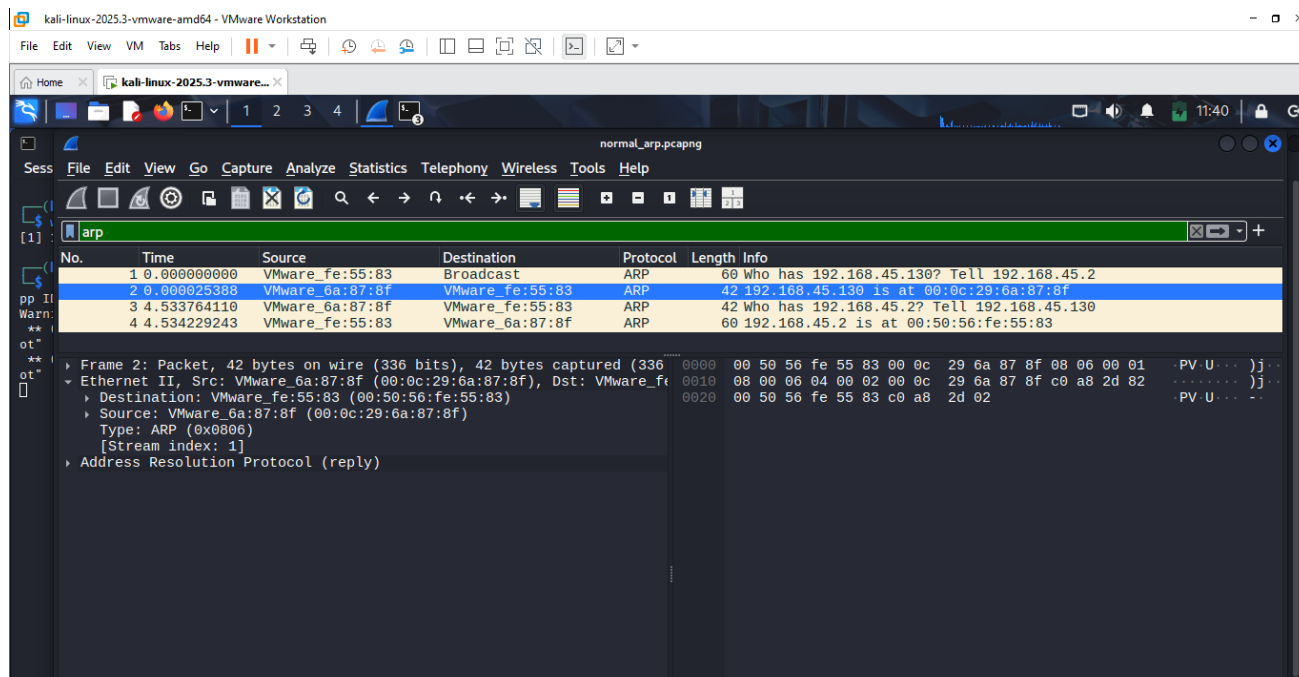


Fig 2.5 Taking record of Frame 2 Ethernet.

Explanation

This screenshot records the Ethernet information for Frame 2, which was the reply to the earlier broadcast request. Unlike the request, the response was directed back to the gateway's MAC address rather than being sent as a broadcast.

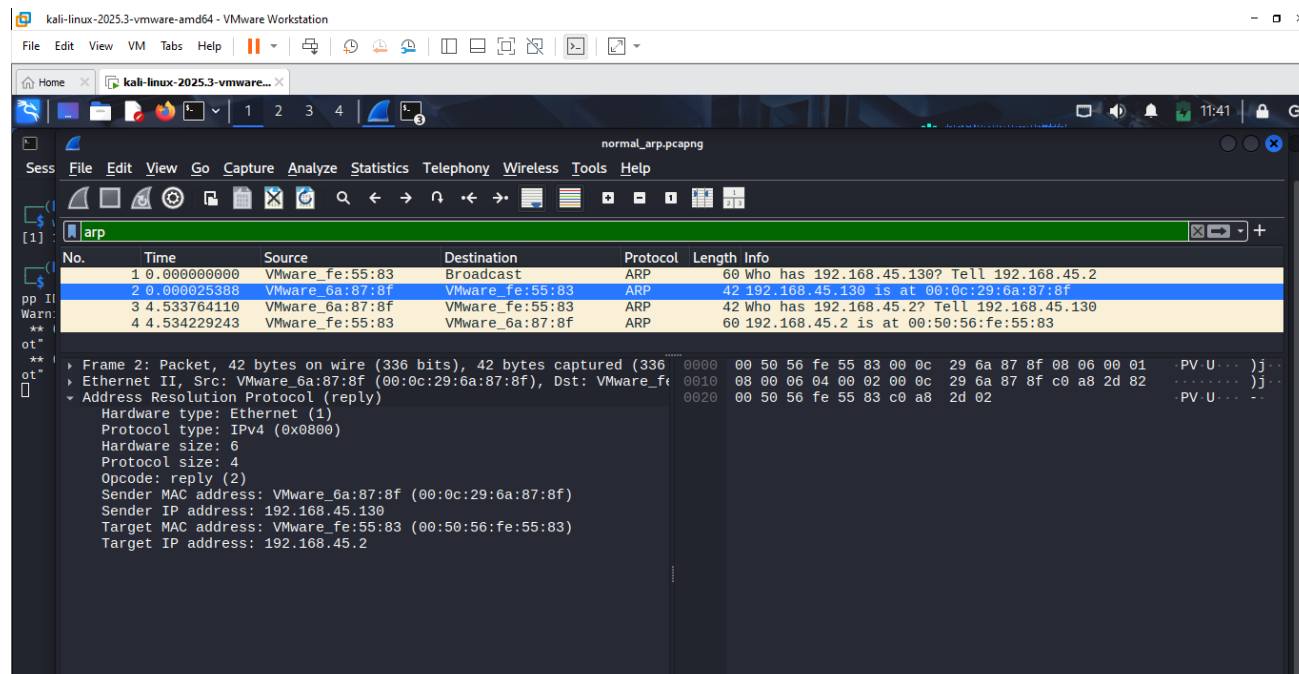
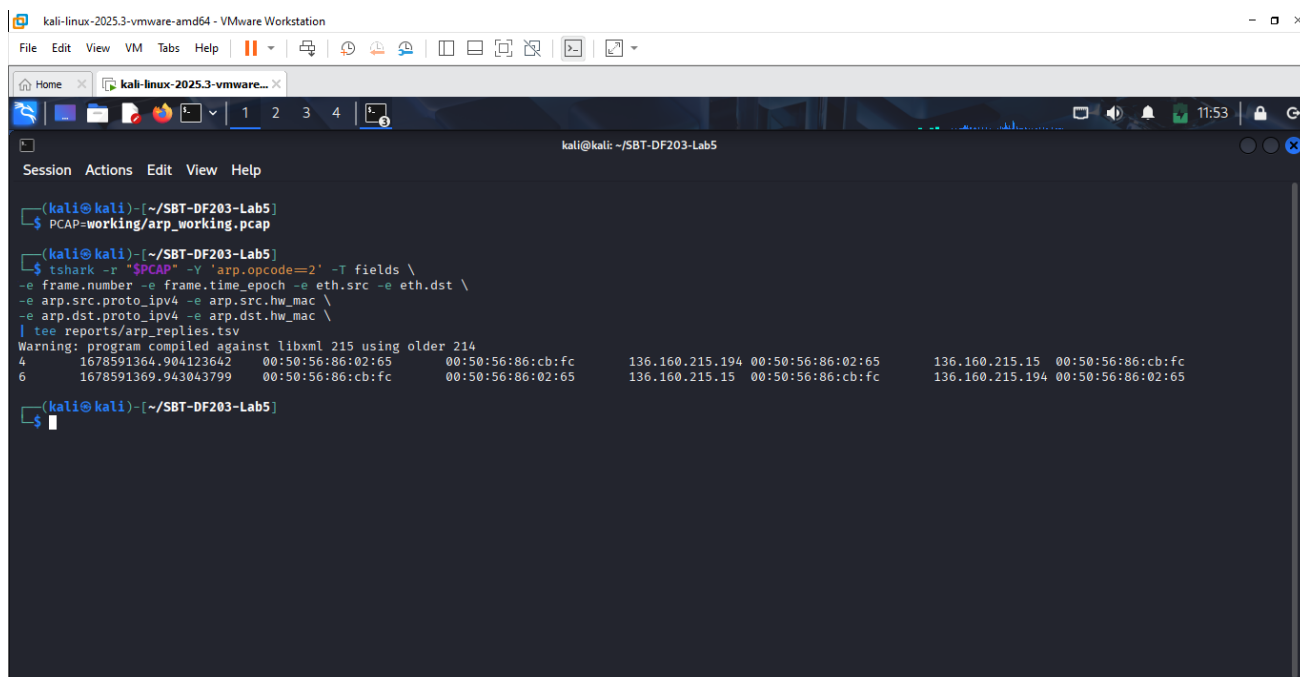


Fig 2.6 Fig 2.4 Taking record of Frame 2 Address Resolution Protocol.

Explanation

This screenshot shows that Frame 2 used ARP opcode 2, identifying it as an ARP reply. Kali reported that 192.168.45.130 was associated with MAC address 00:0c:29:6a:87:8f, providing the information requested by the gateway.



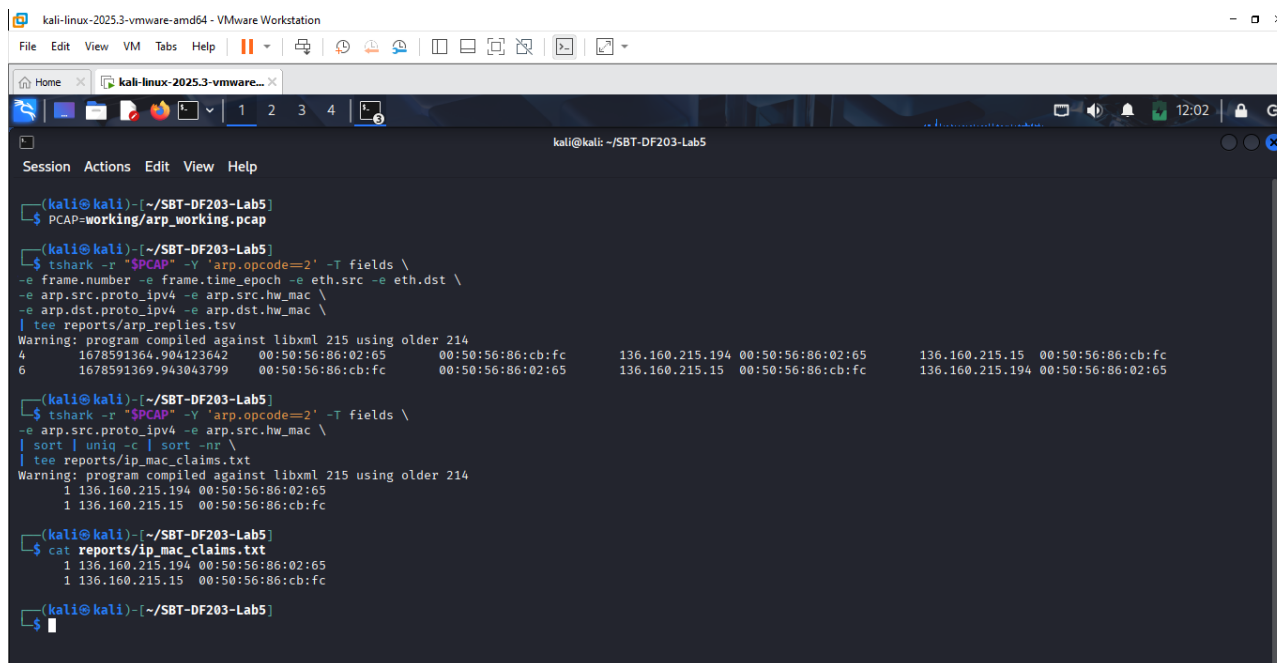
```
kali@kali: ~/SBT-DF203-Lab5
$ PCAP=working/arp_working.pcap
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_replies.tsv
Warning: program compiled against libxml 215 using older 214
4      1678591364.904123642    00:50:56:86:02:65    00:50:56:86:cb:fc    136.160.215.194    00:50:56:86:02:65    136.160.215.15    00:50:56:86:cb:fc
6      1678591369.943043799    00:50:56:86:cb:fc    00:50:56:86:02:65    136.160.215.15    00:50:56:86:cb:fc    136.160.215.194    00:50:56:86:02:65
```

Fig 2.7 Supplied poisoning PCAP.

Frame	Claimed IP	Claimed MAC	Target IP	Target MAC
4	136.160.215.194	00:50:56:86:02:65	136.160.215.15	00:50:56:86:cb:fc
6	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	00:50:56:86:02:65

Explanation

This screenshot shows the ARP replies extracted from the supplied arp.pcap evidence. Frames 4 and 6 were identified for closer analysis because they contained the IP-to-MAC claims exchanged between 136.160.215.194 and 136.160.215.15.



```
kali@kali: ~/SBT-DF203-Lab5
$ PCAP=working/arp_working.pcap

(kali@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e frame.number -e frame.time_epoch -e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_replies.tsv
Warning: program compiled against libxml 215 using older 214
4      1678591364.904123642      00:50:56:86:02:65      136.160.215.194 00:50:56:86:02:65      136.160.215.15  00:50:56:86:02:65
6      1678591369.943043799      00:50:56:86:02:65      136.160.215.15  00:50:56:86:02:65      136.160.215.194 00:50:56:86:02:65

(kali@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr \
| tee reports/ip_mac_claims.txt
Warning: program compiled against libxml 215 using older 214
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15  00:50:56:86:02:65

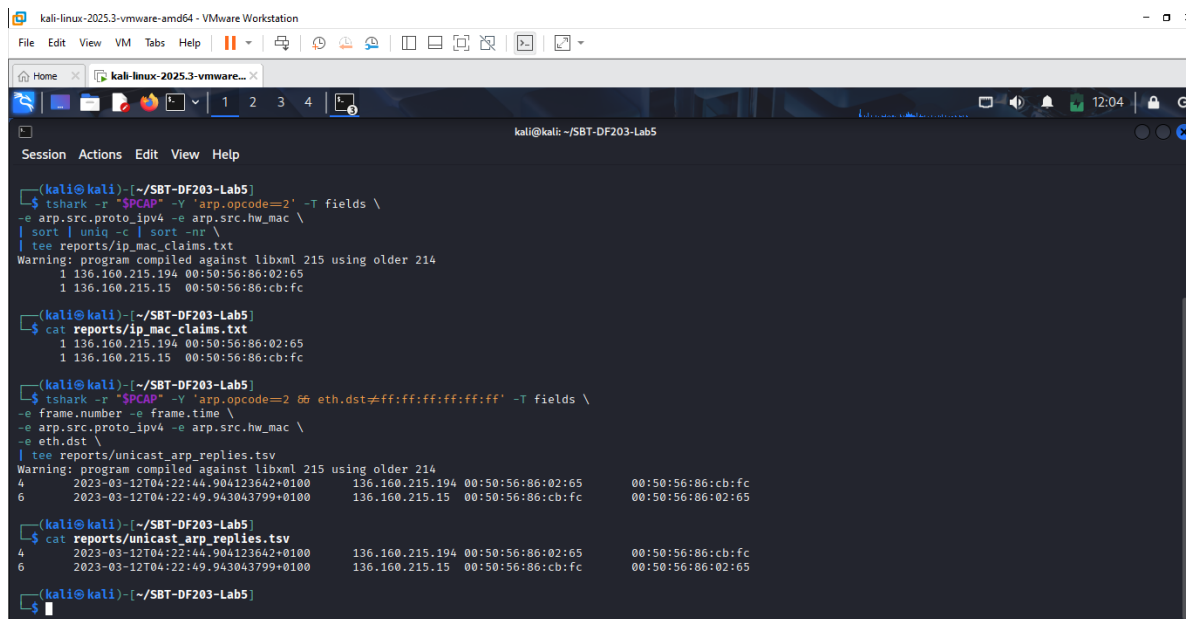
(kali@kali)~/SBT-DF203-Lab5
$ cat reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15  00:50:56:86:02:65

(kali@kali)~/SBT-DF203-Lab5
$
```

Fig 2.8 Supplied poisoning PCAP page 1.

Explanation

This screenshot continues the examination of the supplied ARP capture and provides more detail about the observed packet sequence. It helped me compare the ARP replies with the requests that appeared earlier in the capture.



```
kali@kali: ~/SBT-DF203-Lab5

(kali@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2' -T fields \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
| sort | uniq -c | sort -nr \
| tee reports/ip_mac_claims.txt
Warning: program compiled against libxml 215 using older 214
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15  00:50:56:86:02:65

(kali@kali)~/SBT-DF203-Lab5
$ cat reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15  00:50:56:86:02:65

(kali@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp.opcode==2 && eth.dst==ff:ff:ff:ff:ff:ff' -T fields \
-e frame.number -e frame.time \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e eth.dst \
| tee reports/unicast_arp_replies.tsv
Warning: program compiled against libxml 215 using older 214
4      2023-03-12T04:22:44.904123642+0100      136.160.215.194 00:50:56:86:02:65      00:50:56:86:02:65
6      2023-03-12T04:22:49.943043799+0100      136.160.215.15  00:50:56:86:02:65      00:50:56:86:02:65

(kali@kali)~/SBT-DF203-Lab5
$ cat reports/unicast_arp_replies.tsv
2023-03-12T04:22:44.904123642+0100      136.160.215.194 00:50:56:86:02:65      00:50:56:86:02:65
2023-03-12T04:22:49.943043799+0100      136.160.215.15  00:50:56:86:02:65      00:50:56:86:02:65

(kali@kali)~/SBT-DF203-Lab5
$
```

Fig 2.9 Unicast ARP reply analysis.

Explanation

This screenshot isolates the ARP replies sent directly between hosts rather than broadcast traffic. Reviewing these frames helped determine whether the replies were unsolicited or were responses to legitimate ARP requests.

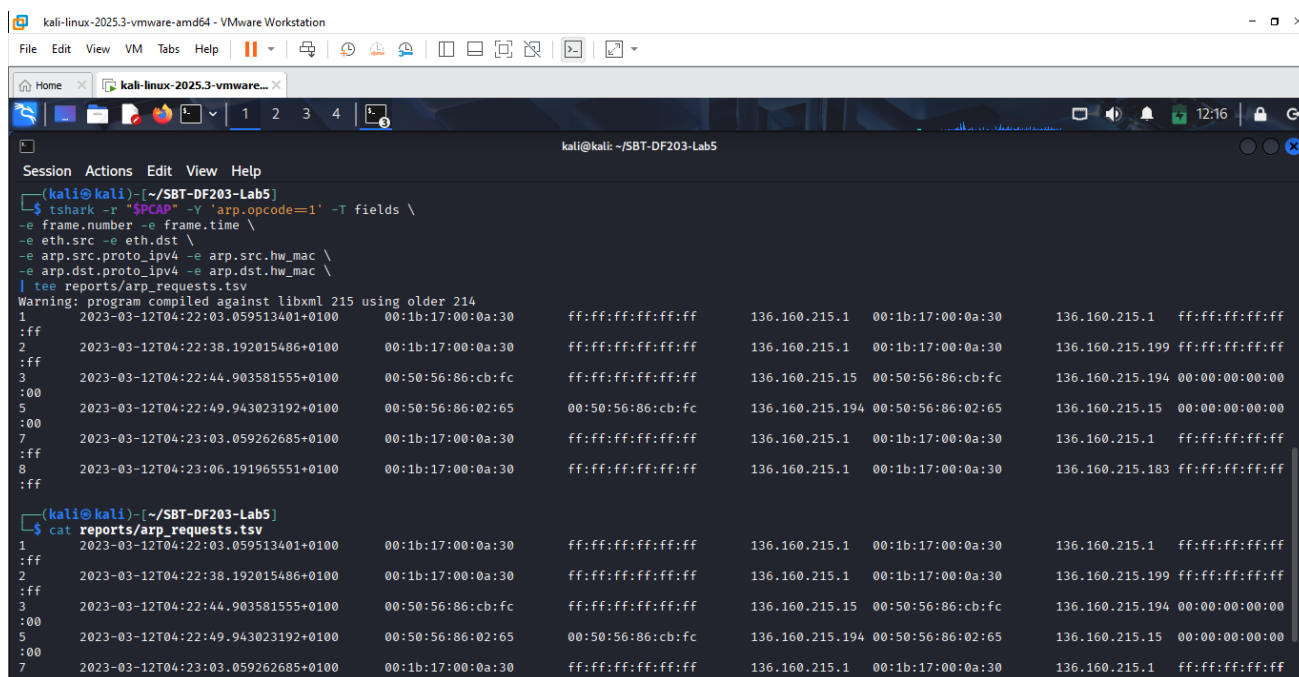


Fig 3.0 Solicited/normal ARP reply.

Explanation

This screenshot shows that the ARP replies could be matched with requests that appeared earlier in the capture. This was important because a reply that follows a corresponding request is consistent with normal ARP behaviour rather than automatically indicating poisoning.

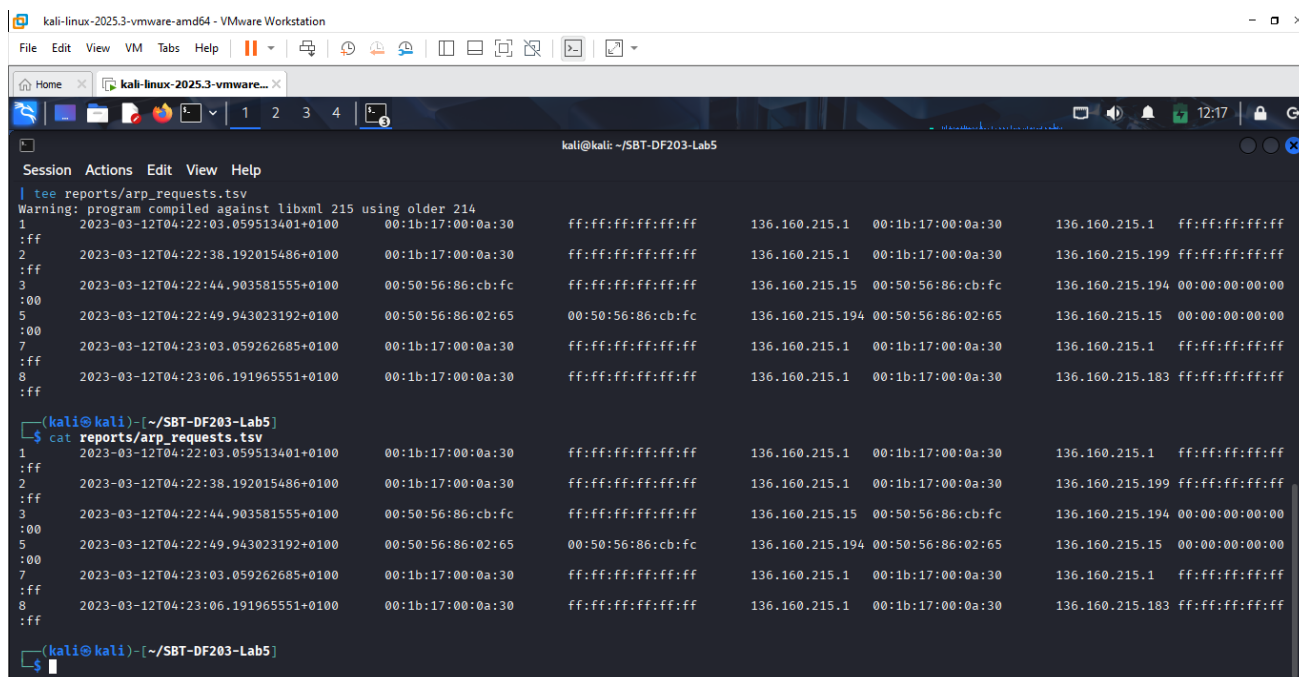


Fig 3.1 Solicited/normal ARP reply page 1.

Reply

Request seen first?

Assessment

Frame 4

Yes — Frame 3

Solicited/normal ARP reply

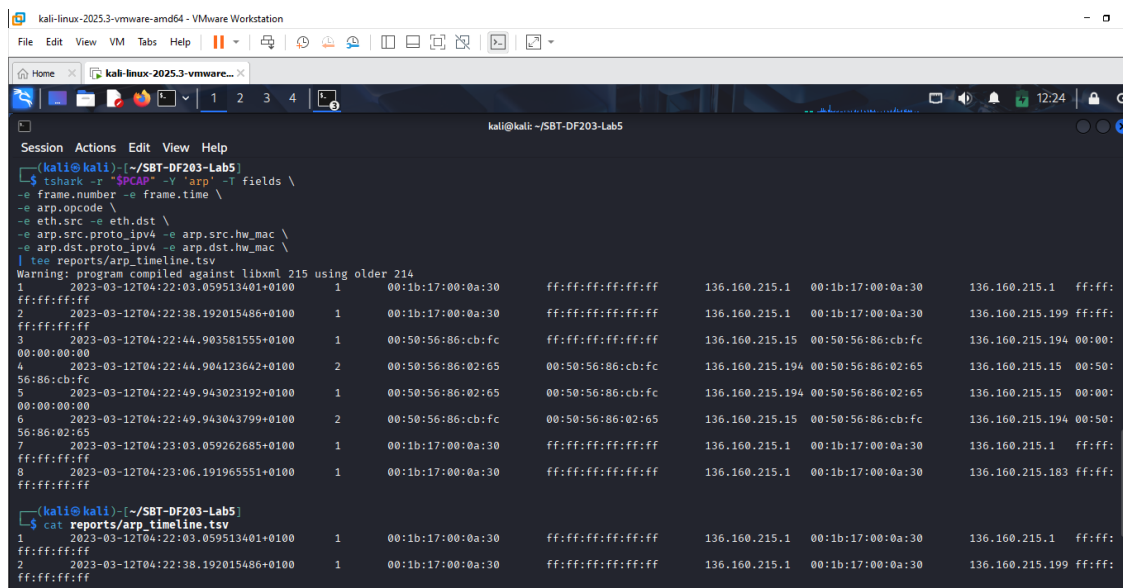
Frame 6

Yes — Frame 5

Solicited/normal ARP reply

Explanation

This screenshot confirms that Frame 4 followed the request in Frame 3, while Frame 6 followed the request in Frame 5. Both replies were therefore classified as solicited ARP replies based on the sequence recorded in the capture.

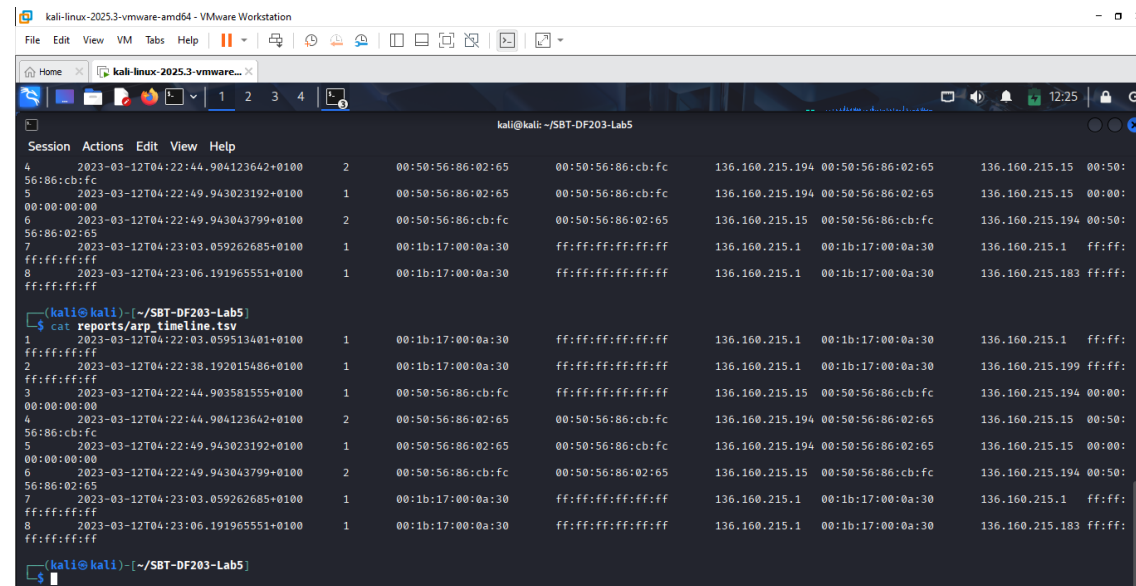


```
kali@kali: ~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp' -T fields \
-e frame.number -e frame.time \
-e arp.opcode \
-e eth.src -e eth.dst \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_timeline.tsv
Warning: program compiled against libxml 215 using older 214
1 2023-03-12T04:22:03.059513401+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-12T04:22:38.192015486+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-12T04:22:44.903581555+0100 1 00:50:56:86:cb:fc ff:ff:ff:ff:ff:ff 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 2023-03-12T04:22:44.904123642+0100 2 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-12T04:22:49.943023192+0100 1 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-12T04:22:49.943043799+0100 2 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-12T04:23:03.059262685+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff
(kali@kali) ~/SBT-DF203-Lab5
$ cat reports/arp_timeline.tsv
1 2023-03-12T04:22:03.059513401+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-12T04:22:38.192015486+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
```

Fig 3.2 Building the full ARP timeline.

Explanation

This screenshot shows the full ARP traffic being extracted in frame order so that the sequence of events could be reconstructed. Building the timeline made it easier to compare each request with the reply that followed it.



```
kali@kali: ~/SBT-DF203-Lab5
$ cat reports/arp_timeline.tsv
4 2023-03-12T04:22:44.904123642+0100 2 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-12T04:22:49.943023192+0100 1 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-12T04:22:49.943043799+0100 2 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-12T04:23:03.059262685+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff
(kali@kali) ~/SBT-DF203-Lab5
$ cat reports/arp_timeline.tsv
1 2023-03-12T04:22:03.059513401+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-12T04:22:38.192015486+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-12T04:22:44.903581555+0100 1 00:50:56:86:cb:fc ff:ff:ff:ff:ff:ff 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 2023-03-12T04:22:44.904123642+0100 2 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-12T04:22:49.943023192+0100 1 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-12T04:22:49.943043799+0100 2 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-12T04:23:03.059262685+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 00:1b:17:00:0a:30 ff:ff:ff:ff:ff:ff 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff
(kali@kali) ~/SBT-DF203-Lab5
```

Fig 3.3 Building the full ARP timeline page 1.

Frame	What happened	Assessment
1	136.160.215.1 broadcasts an ARP request concerning itself	Gratuitous/announcement-style ARP request; not proof of poisoning
2	136.160.215.1 broadcasts asking for 136.160.215.199	Normal ARP request; no reply visible in this capture
3	136.160.215.15 asks for 136.160.215.194	Normal request
4	136.160.215.194 replies with 00:50:56:86:02:65	Solicited reply to Frame 3
5	136.160.215.194 asks for 136.160.215.15	Normal/unicast ARP request
6	136.160.215.15 replies with 00:50:56:86:cb:fc	Solicited reply to Frame 5
7	136.160.215.1 again sends an ARP request concerning itself	Gratuitous/announcement-style request
8	136.160.215.1 broadcasts asking for 136.160.215.183	Normal request; no reply visible in this capture

Explanation

This screenshot presents the interpretation of all eight ARP frames found in the supplied capture. It shows normal requests, two solicited replies, and announcement-style ARP requests involving 136.160.215.1.

```

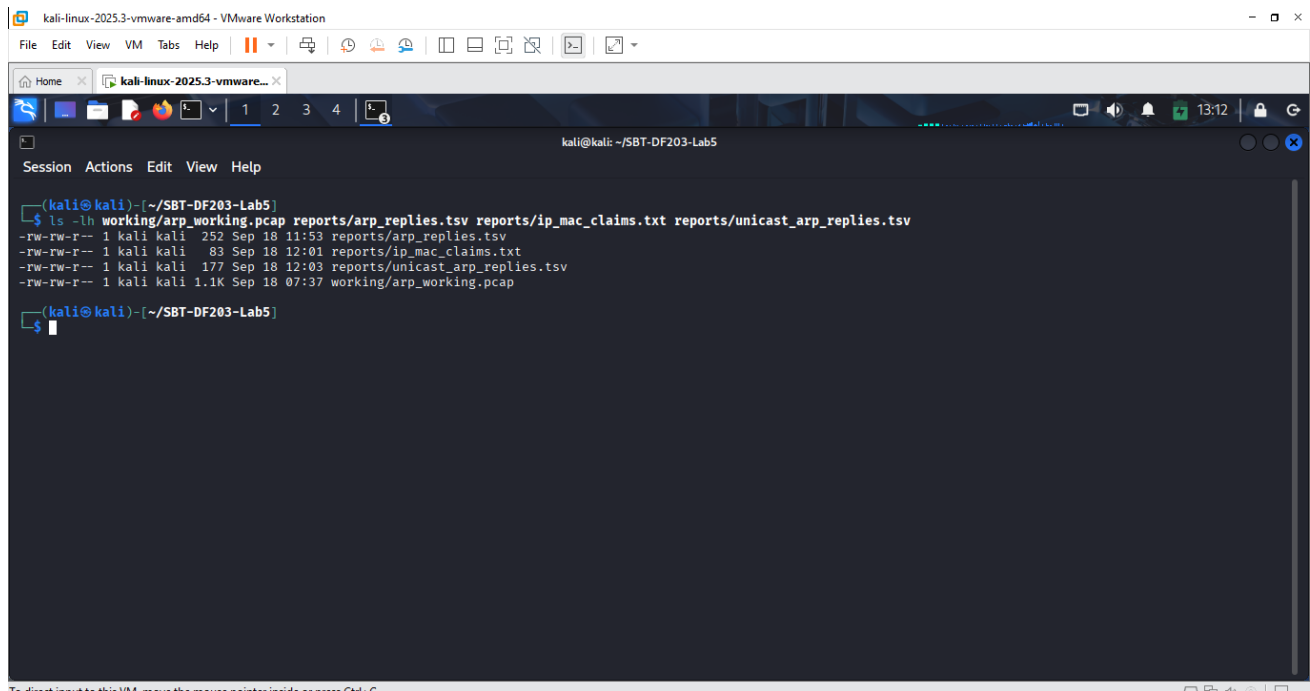
kali-linux-2025.3-vmware-amd64 - VMware Workstation
File Edit View VM Tabs Help
kali-linux-2025.3-vmware...
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
(kali@kali)~/SBT-DF203-Lab5
$ tshark -r "$PCAP" -Y 'arp' -T fields \
  -e arp.src.proto_ipv4 -e arp.src.hw_mac \
  | sort -u \
  | tee reports/all_ip_mac_claims.txt
Warning: program compiled against libxml 215 using older 214
136.160.215.1 00:1b:17:00:0a:30
136.160.215.15 00:50:56:86:cb:fc
136.160.215.194 00:50:56:86:02:65
(kali@kali)~/SBT-DF203-Lab5
$ cat reports/all_ip_mac_claims.txt
136.160.215.1 00:1b:17:00:0a:30
136.160.215.15 00:50:56:86:cb:fc
136.160.215.194 00:50:56:86:02:65
(kali@kali)~/SBT-DF203-Lab5
$

```

Fig 3.4 Checking all ARP sender claims.

Explanation

This screenshot shows the sender IP and MAC information extracted from the ARP packets. Reviewing these claims allowed me to check whether any IP address was being advertised by more than one MAC address.



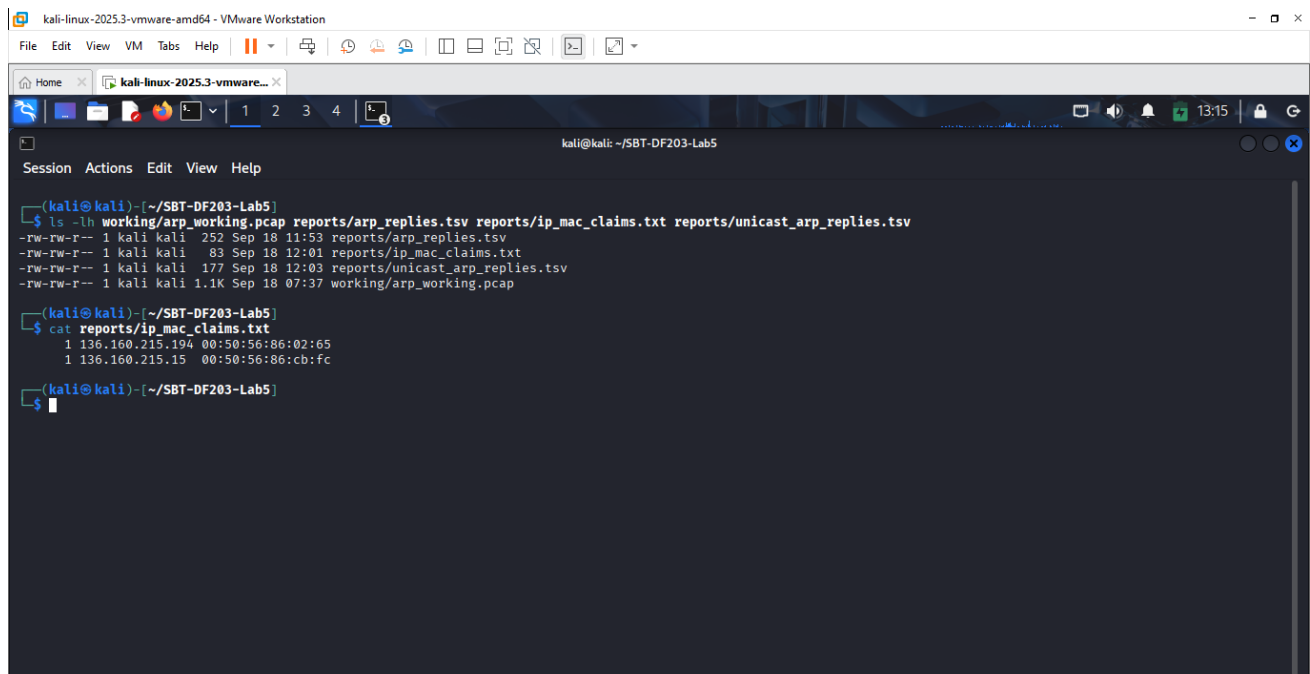
The screenshot shows a terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal prompt is "(kali@kali)-[~/SBT-DF203-Lab5]". The command executed is `ls -lh working/arp_working.pcap reports/arp_replies.tsv reports/ip_mac_claims.txt reports/unicast_arp_replies.tsv`. The output shows four files with their permissions, sizes, dates, and names.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ ls -lh working/arp_working.pcap reports/arp_replies.tsv reports/ip_mac_claims.txt reports/unicast_arp_replies.tsv
-rw-rw-r-- 1 kali kali 252 Sep 18 11:53 reports/arp_replies.tsv
-rw-rw-r-- 1 kali kali 83 Sep 18 12:01 reports/ip_mac_claims.txt
-rw-rw-r-- 1 kali kali 177 Sep 18 12:03 reports/unicast_arp_replies.tsv
-rw-rw-r-- 1 kali kali 1.1K Sep 18 07:37 working/arp_working.pcap
```

Fig 3.5 Verifying Part C evidence.

Explanation

This screenshot was used to confirm that the required Part C analysis files had been generated successfully. It provided a quick check that the ARP reply inventory and related evidence were available for the report.



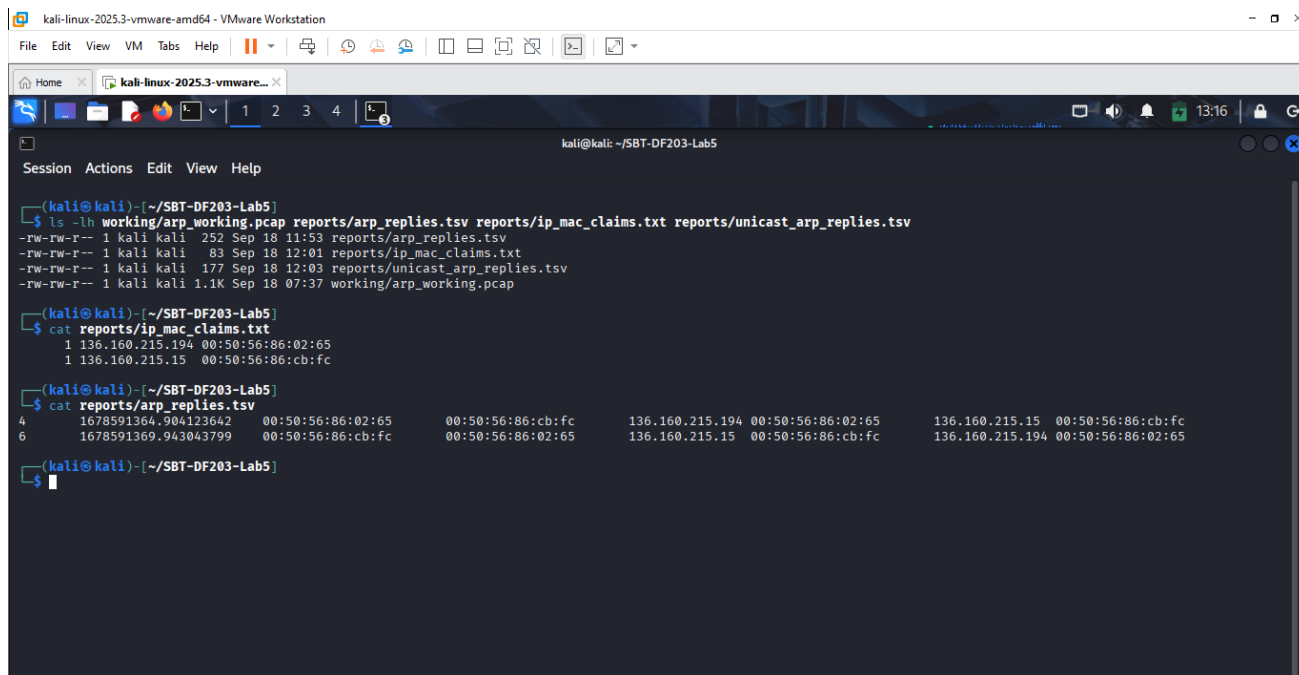
The screenshot shows a terminal window titled "kali-linux-2025.3-vmware-amd64 - VMware Workstation". The terminal prompt is "(kali@kali)-[~/SBT-DF203-Lab5]". The command executed is `cat reports/ip_mac_claims.txt`. The output shows two lines of IP-to-MAC claims.

```
(kali@kali)-[~/SBT-DF203-Lab5]
$ cat reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc
```

Fig 3.6 Inspecting the IP-to-MAC claims.

Explanation

This screenshot shows the IP-to-MAC claims being reviewed from the supplied capture. The purpose was to identify any conflicting mappings that could suggest one IP address was being claimed by different devices.



```
kali@kali: ~/SBT-DF203-Lab5
$ ls -lh working/arp_working.pcap reports/arp_replies.tsv reports/ip_mac_claims.txt reports/unicast_arp_replies.tsv
-rw-rw-r-- 1 kali kali 252 Sep 18 11:53 reports/arp_replies.tsv
-rw-rw-r-- 1 kali kali 83 Sep 18 12:01 reports/ip_mac_claims.txt
-rw-rw-r-- 1 kali kali 177 Sep 18 12:03 reports/unicast_arp_replies.tsv
-rw-rw-r-- 1 kali kali 1.1K Sep 18 07:37 working/arp_working.pcap

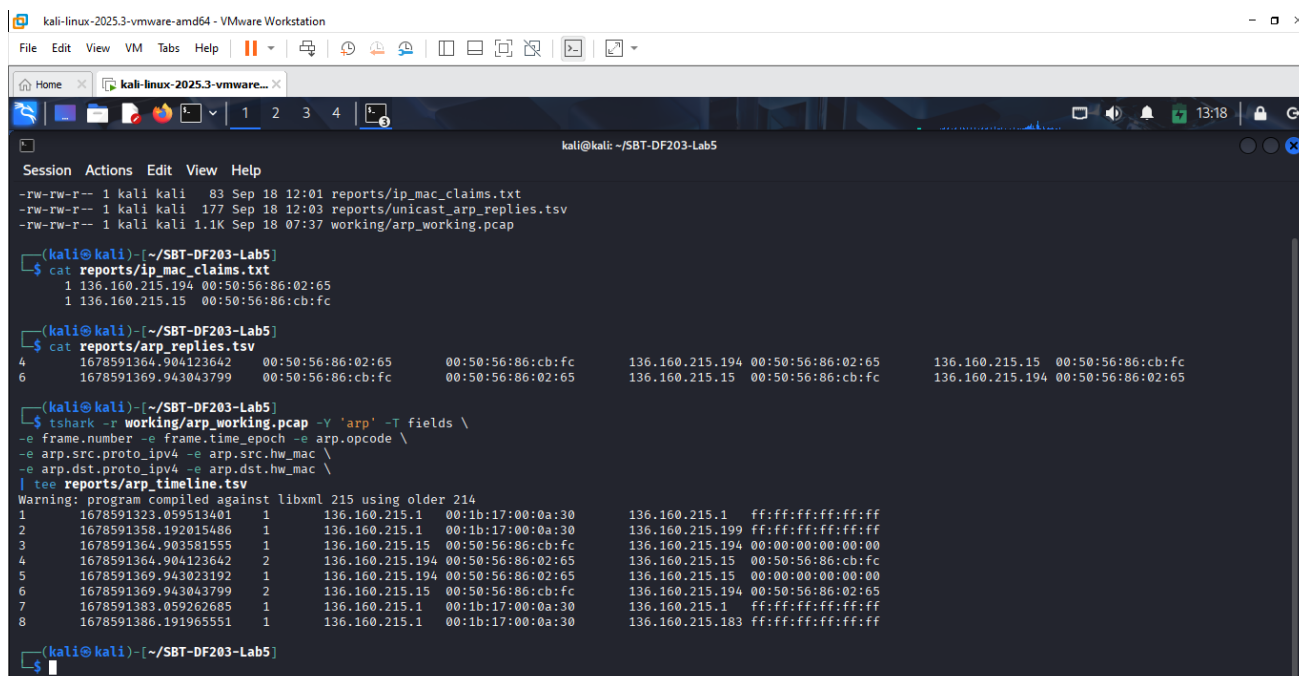
(kali@kali)~/SBT-DF203-Lab5
$ cat reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(kali@kali)~/SBT-DF203-Lab5
$ cat reports/arp_replies.tsv
4 1678591364.904123642 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 1678591369.943043799 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
```

Fig 3.7 Inspecting the IP-to-MAC claims page 1.

Explanation

This screenshot continues the IP-to-MAC examination and shows how often each address pairing appeared in the capture. Comparing the repeated mappings helped determine whether the same IP remained associated with the same MAC address.



```
kali@kali: ~/SBT-DF203-Lab5
$ cat reports/ip_mac_claims.txt
1 136.160.215.194 00:50:56:86:02:65
1 136.160.215.15 00:50:56:86:cb:fc

(kali@kali)~/SBT-DF203-Lab5
$ cat reports/arp_replies.tsv
4 1678591364.904123642 00:50:56:86:02:65 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 1678591369.943043799 00:50:56:86:cb:fc 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65

(kali@kali)~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp' -T fields \
-e frame.number -e frame.time_epoch -e arp.opcode \
-e arp.src.proto_ipv4 -e arp.src.hw_mac \
-e arp.dst.proto_ipv4 -e arp.dst.hw_mac \
| tee reports/arp_timeline.tsv
Warning: program compiled against libxml 215 using older 214
1 1678591323.059513401 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 1678591358.192015486 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 1678591364.903581555 1 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 1678591364.904123642 2 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 1678591369.943023192 1 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 1678591369.943043799 2 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 1678591383.059262685 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 1678591386.191965551 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff
```

Fig 3.8 Building the complete ARP timeline.

Reply frame	Claimed IP	Claimed MAC	Request immediately before?	Assessment
4	136.160.215.194	00:50:56:86:02:65	Yes — frame 3	Solicited/consistent reply
6	136.160.215.15	00:50:56:86:cb:fc	Yes — frame 5	Solicited/consistent reply

Explanation

This screenshot summarizes the two ARP reply frames together with the requests that came immediately before them. Frame 4 followed Frame 3 and Frame 6 followed Frame 5, so both were recorded as solicited and consistent replies.

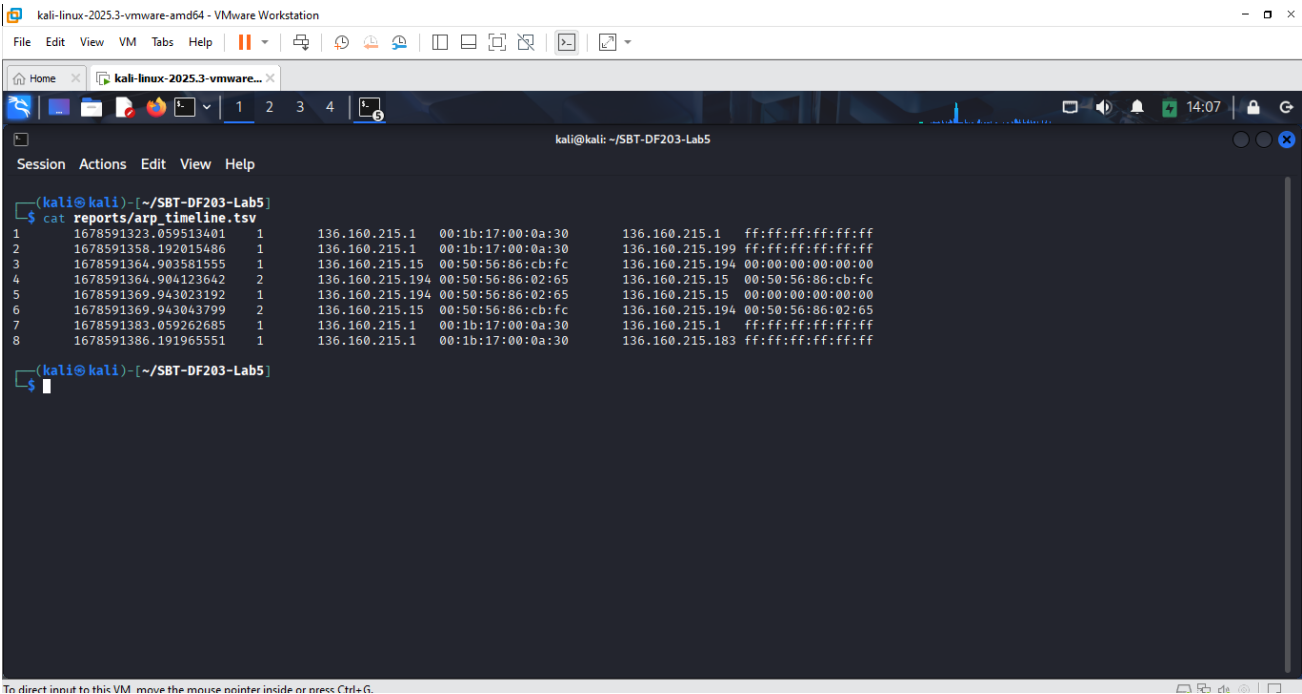


Fig 3.9 Detection Logic and Timeline.

Frame	Opcode	Claimed/Sender IP	Sender MAC	Target IP	Interpretation
1	1	136.160.215.1	00:1b:17:00:0a:30	136.160.215.1	Gratuitous/self-announcement ARP request
2	1	136.160.215.1	00:1b:17:00:0a:30	136.160.215.199	ARP request
3	1	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	Request for .194
4	2	136.160.215.194	00:50:56:86:02:65	136.160.215.15	Reply to frame 3

Explanation

This screenshot organizes the ARP traffic by frame number, opcode, sender IP, sender MAC, target IP, and

interpretation. The timeline helped me distinguish normal requests and replies from announcement-style ARP traffic and check for signs of conflicting claims.

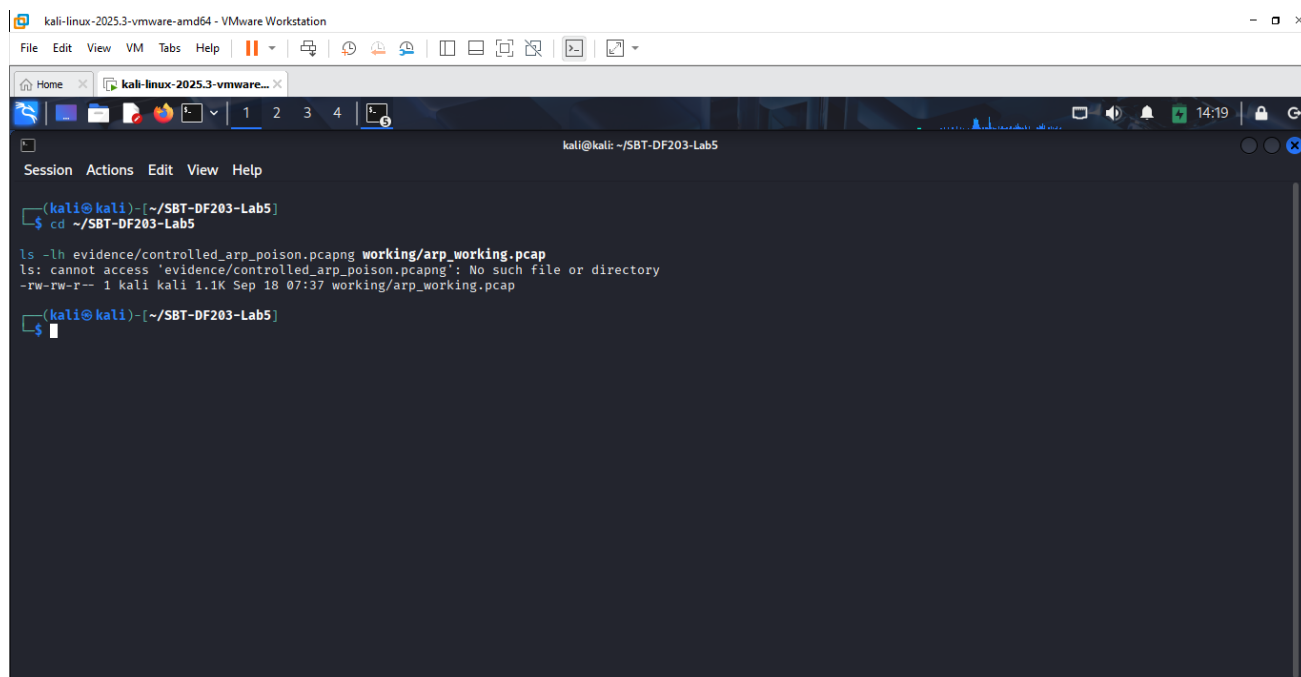


Fig 4.0 Checking all IP-to-MAC claims.

Explanation

This screenshot shows the command used to summarize the IP and MAC address combinations observed in the ARP evidence. The aim was to make any duplicated or conflicting IP-to-MAC claims easier to identify.

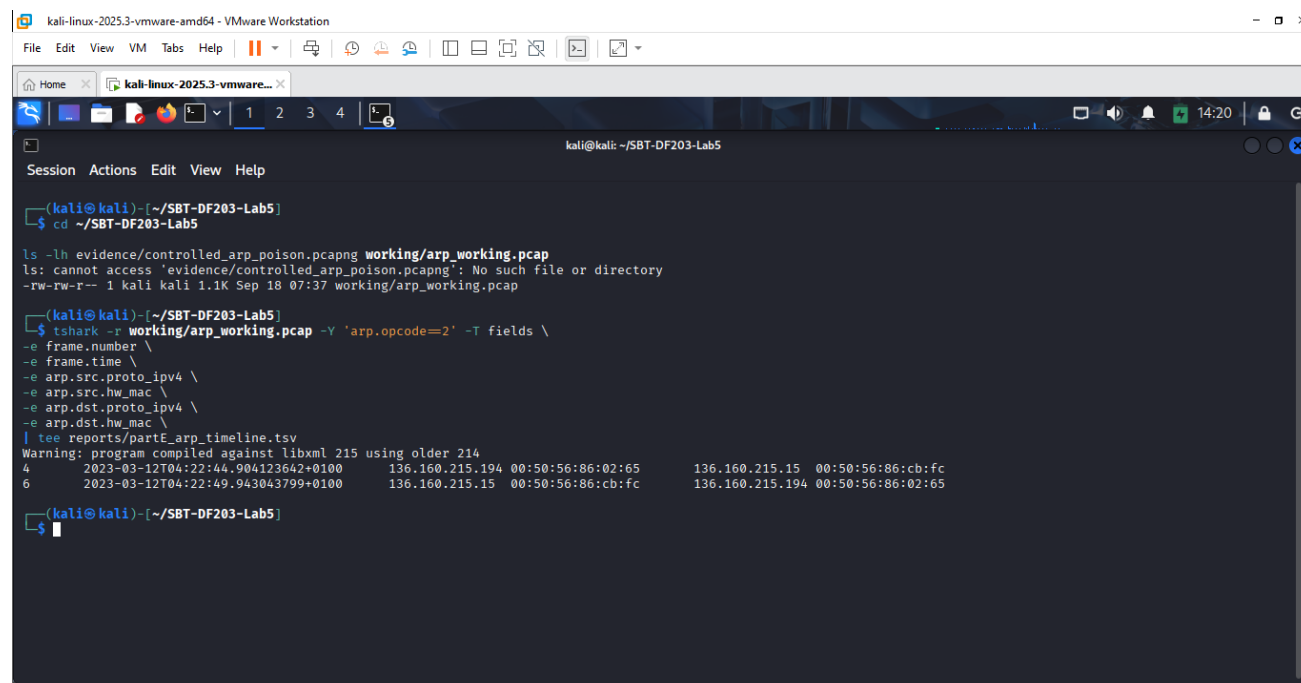
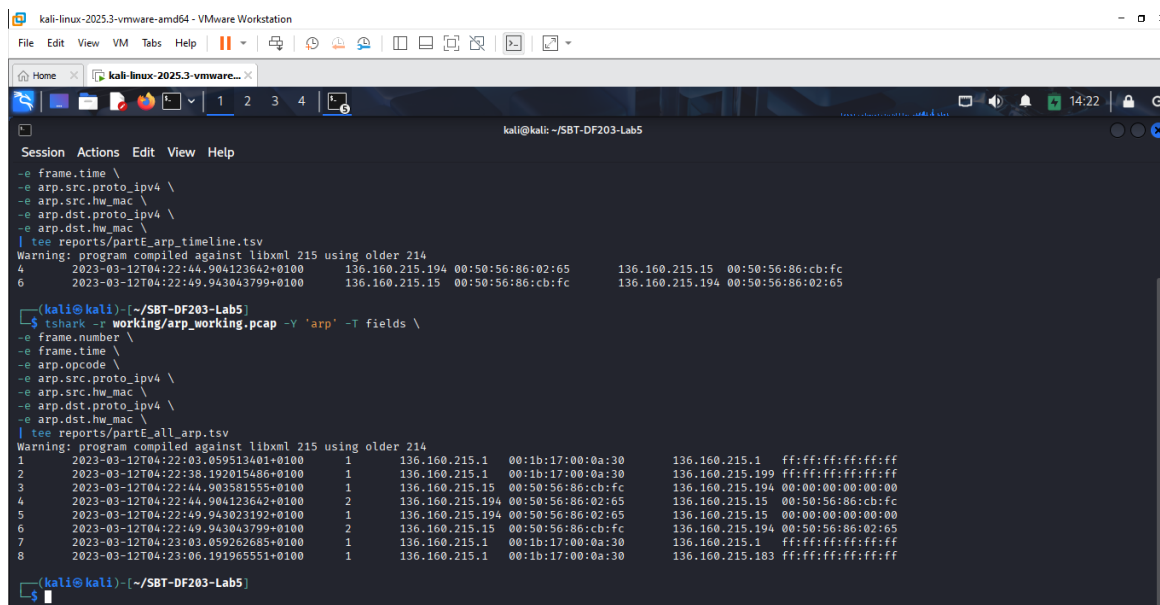


Fig 4.1 Checking all IP-to-MAC claims page 1.

Explanation

This screenshot continues the IP-to-MAC summary and shows the repeated address associations found in the capture. The same pairings continued to appear consistently rather than one IP switching between different MAC addresses.



```
kali@kali: ~/SBT-DF203-Lab5
-e frame.time \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/partE_arp_timeline.tsv
Warning: program compiled against libxml 215 using older 214
4 2023-03-12T04:22:44.904123642+0100 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
6 2023-03-12T04:22:49.943043799+0100 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65

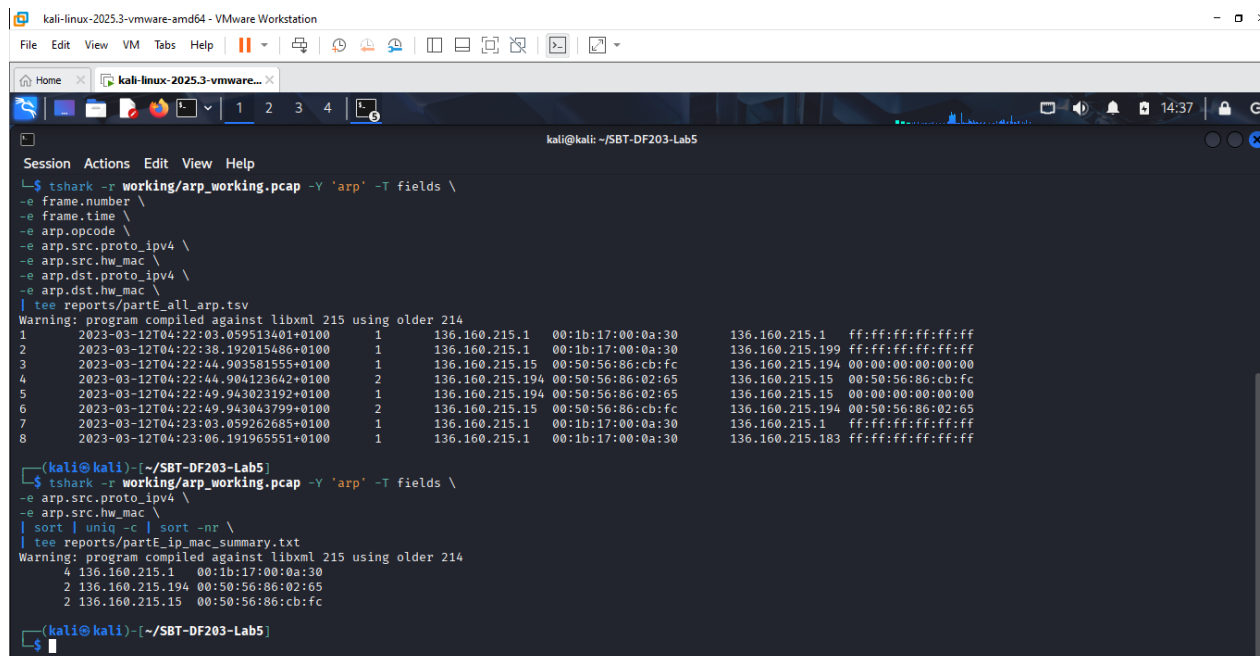
(kali@kali)~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp' -T fields \
-e frame.number \
-e frame.time \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/partE_all_arp.tsv
Warning: program compiled against libxml 215 using older 214
1 2023-03-12T04:22:03.059513401+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-12T04:22:38.192015486+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-12T04:22:44.903581555+0100 1 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 2023-03-12T04:22:44.904123642+0100 2 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-12T04:22:49.943023192+0100 1 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-12T04:22:49.943043799+0100 2 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-12T04:23:03.059262685+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(kali@kali)~/SBT-DF203-Lab5
$
```

Fig 4.2 Checking all IP-to-MAC claims page 2.

Explanation

This screenshot provides another view of the ARP claim results used during the detection stage. It helped confirm that the observed sender mappings were stable across the analyzed packets.



```
kali@kali: ~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp' -T fields \
-e frame.number \
-e frame.time \
-e arp.opcode \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
-e arp.dst.proto_ipv4 \
-e arp.dst.hw_mac \
| tee reports/partE_all_arp.tsv
Warning: program compiled against libxml 215 using older 214
1 2023-03-12T04:22:03.059513401+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-12T04:22:38.192015486+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-12T04:22:44.903581555+0100 1 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 2023-03-12T04:22:44.904123642+0100 2 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-12T04:22:49.943023192+0100 1 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-12T04:22:49.943043799+0100 2 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-12T04:23:03.059262685+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(kali@kali)~/SBT-DF203-Lab5
$ tshark -r working/arp_working.pcap -Y 'arp' -T fields \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
| sort | uniq -c | sort -nr \
| tee reports/partE_ip_mac_summary.txt
Warning: program compiled against libxml 215 using older 214
4 136.160.215.1 00:1b:17:00:0a:30
2 136.160.215.194 00:50:56:86:02:65
2 136.160.215.15 00:50:56:86:cb:fc

(kali@kali)~/SBT-DF203-Lab5
$
```

Fig 4.3 Checking all IP-to-MAC claims page 3.

Count	IP address	MAC address	Interpretation
4	136.160.215.1	00:1b:17:00:0a:30	Consistent mapping
2	136.160.215.194	00:50:56:86:02:65	Consistent mapping
2	136.160.215.15	00:50:56:86:cb:fc	Consistent mapping

The IP-to-MAC summary showed consistent address mappings for 136.160.215.1, 136.160.215.194, and 136.160.215.15. No IP address was observed using more than one MAC address in the analyzed ARP frames. The timeline also showed that Frame 4 followed the request in Frame 3 and Frame 6 followed the request in Frame 5, so both replies were consistent with solicited ARP communication.

Explanation

This screenshot summarizes the three IP-to-MAC relationships found in the evidence. 136.160.215.1, 136.160.215.194, and 136.160.215.15 each remained associated with one consistent MAC address in the analyzed ARP frames.

```

kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
-e arp.dst.hw_mac \
| tee reports/partE_all_arp.tsv
Warning: program compiled against libxml 215 using older 214
1 2023-03-12T04:22:03.059513401+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
2 2023-03-12T04:22:38.192015486+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-12T04:22:44.903581555+0100 1 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 2023-03-12T04:22:44.904123642+0100 2 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-12T04:22:49.943023192+0100 1 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-12T04:22:49.943043799+0100 2 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-12T04:23:03.059262685+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(kali@kali)~[~/SBT-DF203-Lab5]
$ tshark -i working/arp_working.pcap -Y 'arp' -T fields \
-e arp.src.proto.ipv4 \
-e arp.src.hw_mac \
| sort | uniq -c | sort -nr \
| tee reports/partE_ip_mac_summary.txt
Warning: program compiled against libxml 215 using older 214
4 136.160.215.1 00:1b:17:00:0a:30
2 136.160.215.194 00:50:56:86:02:65
2 136.160.215.15 00:50:56:86:cb:fc

(kali@kali)~[~/SBT-DF203-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt

(kali@kali)~[~/SBT-DF203-Lab5]
$ sudo ip neigh flush all
[sudo] password for kali:

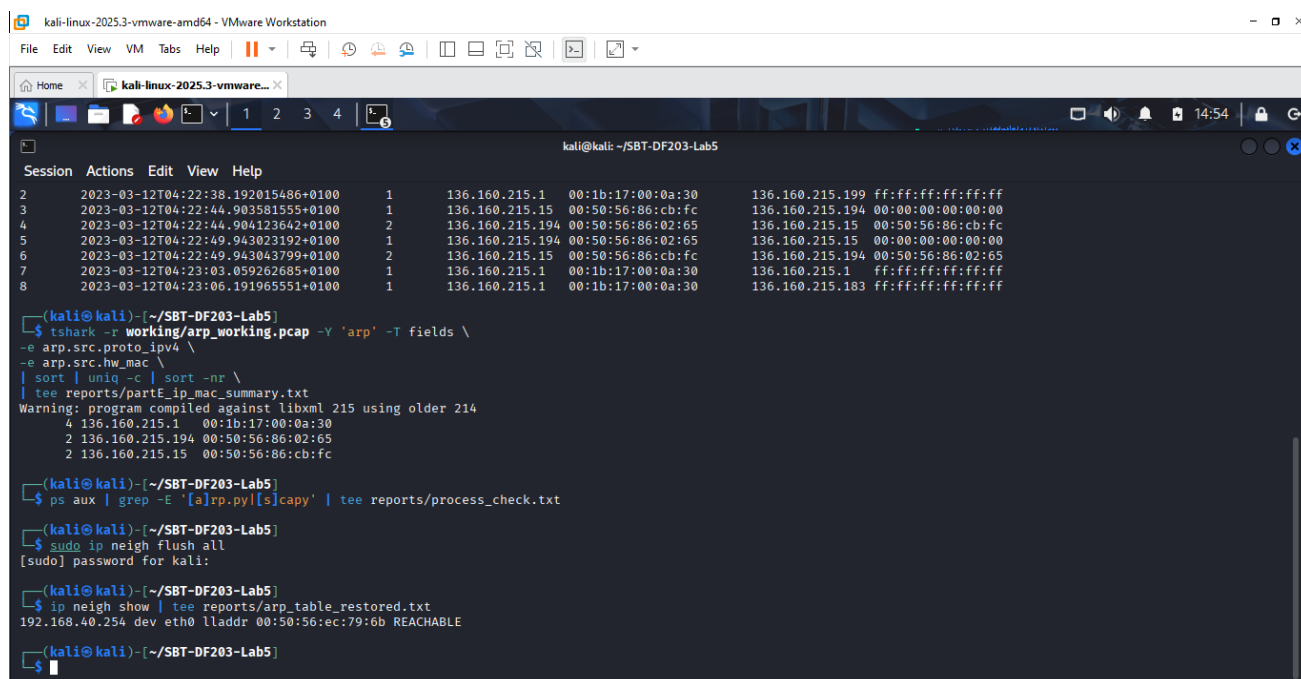
(kali@kali)~[~/SBT-DF203-Lab5]

```

Fig 4.4 Checking all IP-to-MAC claims page 4 and clear the learned ARP/neighbor entries.

Explanation

This screenshot marks the transition from evidence analysis to restoring the lab environment. After completing the IP-to-MAC checks, the learned neighbour entries were cleared so the system could return to a clean ARP state.



```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
2 2023-03-12T04:22:38.192015486+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.199 ff:ff:ff:ff:ff:ff
3 2023-03-12T04:22:44.903581555+0100 1 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:00:00:00:00:00
4 2023-03-12T04:22:44.904723642+0100 2 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:50:56:86:cb:fc
5 2023-03-12T04:22:49.943023192+0100 1 136.160.215.194 00:50:56:86:02:65 136.160.215.15 00:00:00:00:00:00
6 2023-03-12T04:22:49.943043799+0100 2 136.160.215.15 00:50:56:86:cb:fc 136.160.215.194 00:50:56:86:02:65
7 2023-03-12T04:23:03.059262685+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(kali@kali)~-[~/SBT-DF203-Lab5]
$ tshark -r working/arp_working.pcap -Y 'arp' -T fields \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
| sort | uniq -c | sort -nr \
| tee reports/partE_ip_mac_summary.txt
Warning: program compiled against libxml 215 using older 214
 4 136.160.215.1 00:1b:17:00:0a:30
 2 136.160.215.194 00:50:56:86:02:65
 2 136.160.215.15 00:50:56:86:cb:fc

(kali@kali)~-[~/SBT-DF203-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt

(kali@kali)~-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush all
[sudo] password for kali:

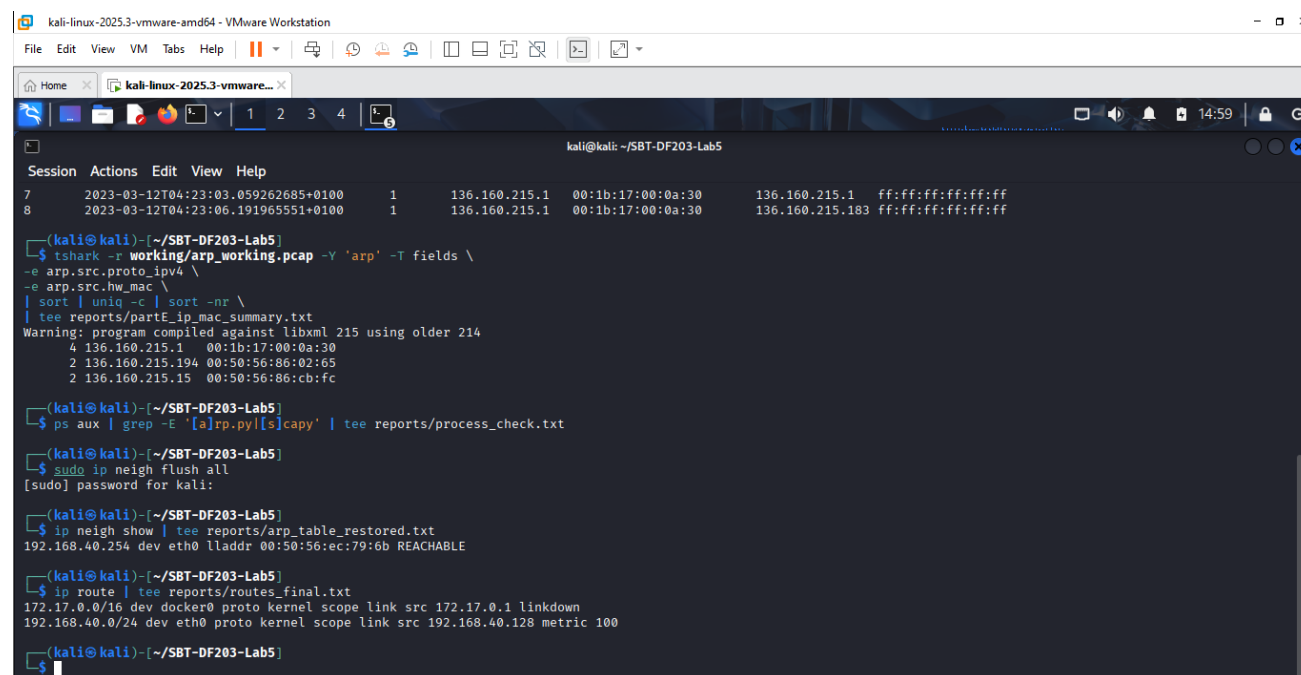
(kali@kali)~-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_restored.txt
192.168.40.254 dev eth0 lladdr 00:50:56:ec:79:6b REACHABLE

(kali@kali)~-[~/SBT-DF203-Lab5]
$
```

Fig 4.5 Verifying the ARP table.

Explanation

This screenshot shows the neighbour table being checked after the learned ARP entries were cleared. The verification was carried out to confirm the state of the ARP cache following the restoration step.



```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
7 2023-03-12T04:23:03.059262685+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.1 ff:ff:ff:ff:ff:ff
8 2023-03-12T04:23:06.191965551+0100 1 136.160.215.1 00:1b:17:00:0a:30 136.160.215.183 ff:ff:ff:ff:ff:ff

(kali@kali)~-[~/SBT-DF203-Lab5]
$ tshark -r working/arp_working.pcap -Y 'arp' -T fields \
-e arp.src.proto_ipv4 \
-e arp.src.hw_mac \
| sort | uniq -c | sort -nr \
| tee reports/partE_ip_mac_summary.txt
Warning: program compiled against libxml 215 using older 214
 4 136.160.215.1 00:1b:17:00:0a:30
 2 136.160.215.194 00:50:56:86:02:65
 2 136.160.215.15 00:50:56:86:cb:fc

(kali@kali)~-[~/SBT-DF203-Lab5]
$ ps aux | grep -E '[a]rp.py|[s]copy' | tee reports/process_check.txt

(kali@kali)~-[~/SBT-DF203-Lab5]
$ sudo ip neigh flush all
[sudo] password for kali:

(kali@kali)~-[~/SBT-DF203-Lab5]
$ ip neigh show | tee reports/arp_table_restored.txt
192.168.40.254 dev eth0 lladdr 00:50:56:ec:79:6b REACHABLE

(kali@kali)~-[~/SBT-DF203-Lab5]
$ ip route | tee reports/routes_final.txt
172.17.0.0/16 dev docker0 proto kernel scope link src 172.17.0.1 linkdown
192.168.40.0/24 dev eth0 proto kernel scope link src 192.168.40.128 metric 100

(kali@kali)~-[~/SBT-DF203-Lab5]
$
```

Fig 4.6 Verifying the ARP table page 1.

Explanation

This screenshot shows the network beginning to learn a neighbour entry again after the ARP cache was cleared. It demonstrates that normal neighbour discovery resumed after the cleanup rather than leaving the previously learned entries unchanged.

```
kali@kali: ~/SBT-DF203-Lab5
Session Actions Edit View Help
} | tee reports/final_audit.txt
===== EVIDENCE =====
total 8.0K
-rw-rw-r-- 1 kali kali 1.1K Sep 18 07:37 arp.pcap
-rw-rw-r-- 1 kali kali 672 Sep 18 10:02 normal_arp.pcapng

===== WORKING =====
total 4.0K
-rw-rw-r-- 1 kali kali 1.1K Sep 18 07:37 arp_working.pcap

===== REPORTS =====
total 76K
-rw-rw-r-- 1 kali kali 99 Sep 18 12:30 all_ip_mac_claims.txt
-rw-rw-r-- 1 kali kali 175 Sep 18 07:38 arp_capture_hashes.txt
-rw-rw-r-- 1 kali kali 252 Sep 18 11:53 arp_replies.tsv
-rw-rw-r-- 1 kali kali 832 Sep 18 12:16 arp_requests.tsv
-rw-rw-r-- 1 kali kali 110 Sep 18 10:05 arp_table_after_ping.txt
-rw-rw-r-- 1 kali kali 114 Sep 18 07:41 arp_table_initial.txt
-rw-rw-r-- 1 kali kali 60 Sep 18 14:54 arp_table_restored.txt
-rw-rw-r-- 1 kali kali 728 Sep 18 13:18 arp_timeline.tsv
-rw-rw-r-- 1 kali kali 252 Sep 18 15:10 final_audit.txt
-rw-rw-r-- 1 kali kali 180 Sep 18 07:41 interfaces.txt
-rw-rw-r-- 1 kali kali 83 Sep 18 12:01 ip_mac_claims.txt
-rw-rw-r-- 1 kali kali 556 Sep 18 10:06 normal_arp_fields.tsv
-rw-rw-r-- 1 kali kali 93 Sep 18 10:11 normal_arp_hash.txt
-rw-rw-r-- 1 kali kali 840 Sep 18 14:22 partE_all_arp.tsv
-rw-rw-r-- 1 kali kali 208 Sep 18 14:20 partE_arp_timeline.tsv
-rw-rw-r-- 1 kali kali 123 Sep 18 14:37 partE_ip_mac_summary.txt
-rw-rw-r-- 1 kali kali 0 Sep 18 14:42 process_check.txt
-rw-rw-r-- 1 kali kali 155 Sep 18 14:58 routes_final.txt
-rw-rw-r-- 1 kali kali 231 Sep 18 07:41 routes.txt
-rw-rw-r-- 1 kali kali 177 Sep 18 12:03 unicast_arp_replies.tsv
```

Fig 4.7 Verifying the ARP table page 2.

Explanation

This screenshot records the final network and routing state after the practical was completed. It provides evidence that the Kali machine remained connected to the isolated lab network after the ARP analysis and restoration steps.

ARP Detection Timeline

The following timeline summarizes the important ARP replies:

Event	Claimed IP	Claimed MAC	Target IP	Request Seen First?	Assessment
Frame 4	136.160.215.194	00:50:56:86:02:65	136.160.215.15	Yes – Frame 3	Normal solicited ARP reply
Frame 6	136.160.215.15	00:50:56:86:cb:fc	136.160.215.194	Yes – Frame 5	Normal solicited ARP reply

The lab manual explains that suspicious behaviour may include repeated replies without matching requests, one MAC claiming several important IP addresses, or a gateway IP changing from its legitimate MAC address to another MAC address.

In the ARP frames I examined, these indicators were not clearly present.

Forensic Finding

The ARP timeline showed that the replies in Frames 4 and 6 both had matching requests before them. Frame 3 requested information about 136.160.215.194, and Frame 4 answered that request. Frame 5 requested information about 136.160.215.15, and Frame 6 provided the corresponding reply.

The IP-to-MAC relationships also remained consistent throughout the analyzed frames.

For this reason, I could not confirm ARP poisoning from these ARP frames alone. Rather than assuming that suspicious activity occurred, I recorded what was actually supported by the available packet evidence.

Impact Assessment

ARP poisoning can be dangerous because ARP does not normally authenticate the information sent in ARP replies.

If an attacker successfully changes a victim's ARP cache, the victim may begin sending traffic to the attacker's MAC address instead of the legitimate destination.

This can allow a man-in-the-middle situation where network traffic may pass through the attacker.

Possible effects can include:

- interception of network traffic;
- redirection of traffic;
- disruption of communication;
- exposure of unencrypted information;
- manipulation of network sessions.

However, these are general consequences of a successful ARP-poisoning attack.

They should not be presented as events that definitely occurred in this particular evidence because the ARP frames analyzed did not conclusively show a conflicting IP-to-MAC mapping.

Restoration of the Lab Environment

After completing the analysis, I checked the system to make sure that no ARP-poisoning or Scapy process was still running.

The following command was used:

```
ps aux | grep -E '[a]rp.py|[s]capy'
```

The command produced no matching output.

This indicated that there was no arp.py or Scapy-based poisoning process running at the time of the check.

I then cleared the learned ARP/neighbor entries using:

```
sudo ip neigh flush all
```

After clearing the entries, I checked the neighbor table again.

The system later showed the following learned local neighbor:

```
192.168.40.254 dev eth0 lladdr 00:50:56:ec:79:6d REACHABLE
```

This showed that Kali had started learning network neighbors again after the ARP table was cleared.

I also checked the final routing table.

The machine was connected to:

```
192.168.40.0/24
```

using:

```
eth0
```

with the source address:

```
192.168.40.128
```

This confirmed that the machine remained on the isolated lab network.

The restoration section of the manual requires learned ARP entries to be cleared and the system to be checked to make sure that no poisoning process remains active.

Preventive Controls

There are several ways ARP-poisoning attacks can be reduced or detected.

One important control is Dynamic ARP Inspection (DAI). On managed switches, DAI can be used together with DHCP snooping to validate ARP messages and reduce the possibility of false ARP information being accepted.

Network administrators can also monitor the MAC address associated with important systems such as the default gateway. A sudden change in the gateway's MAC address may be an indication that something unusual is happening.

Other useful controls include network segmentation, switch port security, and ARP monitoring tools.

Encrypted protocols such as HTTPS and SSH should also be used whenever possible. Even if traffic is intercepted, encryption makes it much more difficult for an attacker to read sensitive information.

These controls are also recommended in the laboratory manual.

Limitations

One limitation of this practical was that a final:

`controlled_arp_poison.pcapng`

file was not available for analysis.

Because of this, the final forensic analysis relied mainly on the supplied:

`arp.pcap`

file and the normal ARP capture I generated during the lab.

Another limitation was that the ARP frames in the supplied capture did not show an IP address being associated with more than one MAC address.

The replies that were examined also had matching requests before them.

For that reason, it would not be appropriate to claim that the analyzed ARP frames conclusively proved an ARP-poisoning attack.

Instead, the report records the evidence exactly as it appeared during the investigation.

Conclusion

This practical gave me a clearer understanding of how ARP works and how ARP traffic can be examined during a digital forensic investigation.

I was able to capture normal ARP traffic, identify ARP requests and replies, examine important packet fields, compare IP-to-MAC address mappings, and build a basic ARP timeline.

The supplied packet capture showed communication between 136.160.215.15 and 136.160.215.194. The replies observed in Frames 4 and 6 both had matching requests before them, while the IP-to-MAC mappings remained consistent in the analyzed traffic.

Based on the evidence available, I could not conclusively identify an ARP-poisoning event from those ARP frames alone.

This was an important part of the exercise because forensic analysis should be based on what the evidence actually shows rather than what the investigator expects to find.

At the end of the practical, I confirmed that no ARP-poisoning process was still running, cleared the learned neighbor entries, and checked the final network state.

Overall, the lab demonstrated how packet captures, ARP tables, IP-to-MAC relationships, timestamps, and system information can be combined to investigate possible ARP-related attacks.

Screenshots

- Fig 1.0 Lab folder structure.
- Fig 1.1 Confirming all package are installed.
- Fig 1.2 Updating the applications.
- Fig 1.3 Updating the applications page 1.
- Fig 1.4 Updating the applications page 2.
- Fig 1.5 Acquiring and preserve and hashing the supplied ARP evidence.
- Fig 1.6 Recording the initial network state.
- Fig 1.7 Recording the Mini Evidence / Chain-of-Custody information.
- Fig 1.8 Normal ARP Resolution.
- Fig 1.9 Capturing normal ARP resolution and clearing only that gateway entry.
- Fig 2.0 Capture succeeded.
- Fig 2.1 Capture succeeded page 1.
- Fig 2.1 Calculating the hash and confirming the evidence outputs.
- Fig 2.2 Opening the capture with wireshark.
- Fig 2.3 Taking record of Frame 1 Ethernet.
- Fig 2.4 Taking record of Frame 1 Address Resolution Protocol.
- Fig 2.5 Taking record of Frame 2 Ethernet.
- Fig 2.6 Fig 2.4 Taking record of Frame 2 Address Resolution Protocol.
- Fig 2.7 Supplied poisoning PCAP.
- Fig 2.8 Supplied poisoning PCAP page 1.
- Fig 2.9 Unicast ARP reply analysis.
- Fig 3.0 Solicited/normal ARP reply.
- Fig 3.1 Solicited/normal ARP reply page 1.
- Fig 3.2 Building the full ARP timeline.
- Fig 3.3 Building the full ARP timeline page 1.
- Fig 3.4 Checking all ARP sender claims.
- Fig 3.5 Verifying Part C evidence.
- Fig 3.6 Inspecting the IP-to-MAC claims.
- Fig 3.7 Inspecting the IP-to-MAC claims page 1.
- Fig 3.8 Building the complete ARP timeline.
- Fig 3.9 Detection Logic and Timeline.
- Fig 4.0 Checking all IP-to-MAC claims.
- Fig 4.1 Checking all IP-to-MAC claims page 1.
- Fig 4.2 Checking all IP-to-MAC claims page 2.
- Fig 4.3 Checking all IP-to-MAC claims page 3.
- Fig 4.4 Checking all IP-to-MAC claims page 4 and clear the learned ARP/neighbor entries.
- Fig 4.5 Verifying the ARP table.
- Fig 4.6 Verifying the ARP table page 1.
- Fig 4.7 Verifying the ARP table page 2.

References

ICDFA Materials.